This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Large Language Model-Assisted Evolutionary Neural Architecture Search for Cognitive Diagnosis

Shangshang Yang, *Member, IEEE*, Linrui Qin, Xiaoshan Yu, Ziwen Wang, Ye Tian, *Senior Member, IEEE*, Xueming Yan, *Senior Member, IEEE*, Haiping Ma, and Xingyi Zhang, *Fellow, IEEE*

***Abstract*—Neural architecture search (NAS) has made remarkable progress in various field tasks and is also used in fields like cognitive diagnosis (CD) in intelligent education. Due to the optimization challenges of the tree-like and variable-length model topology raised by the search space, existing NAS for CD intuitively uses evolutionary algorithms and empirically develops a genetic operator based on basic tree modifications. Nevertheless, the operator's stochastic nature may impede rapid convergence, as the choice of modifications is random and ignores parent information, thereby failing to generate effective models from most parents. To this end, this paper proposes a novel evolutionary NAS approach for CD based on a tailored genetic operator, where the operator incorporates powerful large language models (LLMs) as its key components for tree-encoded model generation. Specifically, three roles of LLMs are defined in the LLM-assisted operator through structured prompt engineering that explicitly defines the LLM's role, constrains the architecture representation, standardizes the output format, and incorporates Hall-of-Fame guidance. The first LLM focuses on exploitation, enhancing model performance through minor incremental adjustments to existing architectures, the second emphasizes exploration, introducing innovative models through creative combinations that have not been previously considered, and the third aims at balancing the contributions of the first two, ensuring a harmonious integration of exploitation and exploration. In each generation, only one role of LLMs is used through an LLM role-switching strategy. Additionally, the empirically-built genetic operator is also employed for partial models' generation for better diversity. Experiments on several datasets show the performance superiority of models searched by the proposed approach over handcrafted models and automatically-designed models. Besides, the effectiveness of the LLM-assisted operator in accelerating algorithm convergence is validated.**

***Index Terms*—Large language models (LLMs), cognitive diagnosis (CD), neural architecture search (NAS), evolutionary algorithm, tree-encoding.**

I. Introduction

AS A cornerstone in automated machine learning, neural architecture search (NAS) [1], [2] aims to automate the design of optimal deep neural network (DNN) architectures, significantly reducing human labor. To achieve this, the architecture search task is generally formulated as an optimization problem that seeks to maximize model performance while commonly minimizing model complexity [3]. To tackle this problem, various optimization strategies have been developed, such as reinforcement learning [4], gradient-based optimization [5], and evolutionary algorithms (EAs) [6]. Through the exploration of the vast architecture search space using these strategies, NAS has achieved remarkable success in a wide range of applications, including image classification [2] and object detection [8] in computer vision (CV), translation [7] and text summarization [9] in natural language processing (NLP), speech recognition [10] in the audio domain, and many others [11], [12].

Recently, cognitive diagnosis (CD) [13], as a tool to estimate students' proficiency in knowledge concepts by analyzing their historical learning records, has gained significant attention in intelligent education due to its provided convenience for subsequent recommendations [14], [15]. To make the diagnosis results convincing, traditional cognitive diagnosis models (CDMs), including IRT [16], DINA [17], NCD [18], and their variants [19], [20], are designed with a strong reliance on the theory of educational psychology and human expertise. Consequently, these CDMs typically feature simple architectures that offer high model interpretability, but their performance tends to be limited when applied to complex and large-scale learning data [21].

Given significant strides made by NAS in other domains, researchers have recently begun exploring NAS techniques for CD, termed NAS-CD approaches [22], [23]. Specifically, EMO-NAS-CD [23] introduces a tree-like search space to represent

Received 12 September 2025; revised 12 January 2026; accepted 13 March 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62302010, Grant 62522301, Grant W2441019, Grant 62136008, Grant 62276001, and Grant 62576116, in part by the Postdoctoral Fellowship Program (Grade B) of China Postdoctoral Science Foundation under Grant GZB20240002, in part by Anhui Provincial Natural Science Foundation under Grant 2308085J03, and in part by the Anhui Province Key Laboratory of Intelligent Computing and Applications under Grant AFZNJS2024KF01. *(Corresponding author: Ye Tian.)*

Shangshang Yang is with the Key Laboratory of Intelligent Computing and Signal Processing of Ministry of Education, the School of Computer Science and Technology, Anhui University, Hefei 230601, China, and also with the Anhui Province Key Laboratory of Intelligent Computing and Applications, Huaibei Normal University, Huaibei 235000, China (e-mail: yangshang0308@gmail.com).

Linrui Qin and Xiaoshan Yu are with the School of Artificial Intelligence, Anhui University, Hefei 230601, China (e-mail: stonewallqr@gmail.com; yxsleo@gamil.com).

Ziwen Wang, Ye Tian, and Xingyi Zhang are with the Key Laboratory of Intelligent Computing and Signal Processing of Ministry of Education, School of Computer Science and Technology, Anhui University, Hefei 230601, China (e-mail: wzw12sir@gmail.com; field910921@gmail.com; xyzhanghust@gmail.com).

Xueming Yan is with the School of Information Science and Technology, Guangdong University of Foreign Studies, Guangzhou 510006, China (e-mail: yanxm@gdufs.edu.cn).

Haiping Ma is with the Information Materials and Intelligent Sensing Laboratory of Anhui Province, Institutes of Physical Science and Information Technology, Anhui University, Hefei 230601, China (e-mail: hpma@ahu.edu.cn).

Recommended for acceptance by Y. Hou.

Digital Object Identifier 10.1109/TETCI.2026.3683785

2471-285X © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Anhui University. Downloaded on October 26,2026 at 14:55:52 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

CDMs as computational trees and searches interpretable and effective models through a multi-objective evolutionary algorithm (MOEA) framework [24]. Its genetic operator is empirically constructed using four basic tree modifications in genetic programming (GP) [25], [26]. As a follow-up work, NAS-GCD [22] develops a deduplication strategy to avoid redundant evaluations of tree-encoded models but still uses the same search space and operator. Despite the promising results achieved by these NAS-CD approaches, the application of NAS in CD remains in its infancy regarding the search algorithm design, especially for the empirically-built genetic operator [19]. The operator randomly applies basic tree modifications (i.e., exchange, replace, delete, and insert) without considering the specific structure or properties of parents. This lack of informed guidance and the stochastic nature often result in ineffective offspring models, both in terms of model performance and architectural innovation, thereby hindering rapid convergence of the search process [27].

In this context, large language models (LLMs) have achieved remarkable success across a wide range of domains [28], thanks to their exceptional abilities to understand, generate, and manipulate complex structures [29]. For example, LLMs have revolutionized tasks in NLP such as text generation, summarization, and machine translation by effectively capturing semantic and contextual information [30]; Besides, LLMs have been applied successfully in tasks like code generation [31], scientific discovery [32], and decision-making [33], showcasing their versatility and adaptability across domains. Building on these strengths, LLMs offer significant potential to address the limitations of existing NAS-CD approaches, particularly in the design of genetic operators [34]. Unlike traditional stochastic operators that rely on random modifications without contextual awareness, LLMs can analyze parent models, assess their structures, and generate offspring that are both innovative and effective. Furthermore, recent pioneering efforts [35], [36] that leverage LLMs to assist evolutionary algorithms (EAs) and NAS demonstrate their feasibility and inspire us to integrate LLMs into NAS-CD for more effective and efficient search processes.

To this end, this paper proposes a novel evolutionary NAS approach for CD that incorporates powerful LLM as key components of a tailored genetic operator. This approach, called LLM-NAS-CD, leverages the capabilities of LLMs to generate effective architectures for CD, thereby enhancing the search process. Specifically, the contributions of this paper are as follows:

- This paper proposes a novel LLM-assisted genetic operator, which defines three roles for LLMs, i.e., exploitation, exploration, and balancing, through structured prompt engineering for targeted and effective modifications of model architectures. The exploitation-focused LLM refines existing architectures with minor incremental adjustments, the exploration-focused LLM generates novel architectures with innovative combinations, and the balancing-focused LLM harmonizes both to achieve the exploration and the exploitation simultaneously to ensure consistent progress.
- This paper develops the LLM-NAS-CD framework, which dynamically selects one role of LLMs at each generation for offspring model generation through an LLM role-switching strategy. This strategy determines the population states by observing the population improvement for adaptively achieving the balance between exploitation and exploration. Additionally, the framework incorporates the traditional empirically-built genetic operator for partial model generation, maintaining diversity within the population.
- Comprehensive experiments on five datasets demonstrate the superior performance of the proposed approach compared to handcrafted and automatically-designed models. The results also validate the effectiveness of the LLM-assisted operator in accelerating algorithm convergence.

II. Preliminaries and Related Work

A. The Cognitive Diagnosis Task

The cognitive diagnosis task typically involves three entities: N students, M exercises, and K knowledge concepts, denoted as $S = \{s_1, s_2, \ldots, s_N\}$, $E = \{e_1, e_2, \ldots, e_M\}$, and $C = \{c_1, c_2, \ldots, c_K\}$, respectively. The relationship between exercises and concepts is usually encoded by a binary matrix $Q \in \{0,1\}^{M\times K}$, known as the Q-matrix, where $Q_{jk} = 1$ indicates that exercise e_j is associated with concept c_k, and $Q_{jk} = 0$ otherwise. Students' learning behaviors are collected in the response logs $R_{log} = \{(s_i, e_j, r_{ij})\}$, where $s_i \in S$, $e_j \in E$, and $r_{ij} \in \{0,1\}$ represents the response of student s_i on exercise e_j. Here, $r_{ij} = 1$ means a correct response, while $r_{ij} = 0$ indicates an incorrect one.

Definition: Given R_{log} and the Q-matrix, the goal of CD is to infer students' mastery of knowledge concepts by modeling their response patterns. To achieve this, a cognitive diagnosis model $\mathcal{F}(\cdot)$ is constructed to predict whether a student can correctly answer an exercise, and the latent student vector learned from $\mathcal{F}(\cdot)$ is taken as an indicator of knowledge proficiency. Concretely, to predict the score of student s_i on exercise e_j, the model $\mathcal{F}(\cdot)$ receives three types of input features: the student embedding $\mathbf{h}_S \in \mathbb{R}^{1\times D}$, the exercise embedding $\mathbf{h}_E \in \mathbb{R}^{1\times D}$, and the concept-related vector $\mathbf{h}_C \in \mathbb{R}^{1\times K}$ derived from Q. These representations are obtained as:

$$\begin{aligned} &\mathbf{h}_S = \mathbf{x}_i^S W_S, \quad \mathbf{h}_E = \mathbf{x}_j^E W_E, \\ &\mathbf{h}_C = \mathbf{x}_j^E Q = (Q_{j1}, Q_{j2}, \ldots, Q_{jK}), \end{aligned} \tag{1}$$

where D is the embedding dimension (typically set equal to K), $\mathbf{x}_i^S \in \{0,1\}^{1\times N}$ and $\mathbf{x}_j^E \in \{0,1\}^{1\times M}$ are one-hot vectors for student s_i and exercise e_j, and $W_S \in \mathbb{R}^{N\times D}$ and $W_E \in \mathbb{R}^{M\times D}$ are trainable embedding matrices. Finally, the CDM $\mathcal{F}(\cdot)$ outputs the predicted probability of correctness as

$$\hat{r}_{ij} = \mathcal{F}(\mathbf{h}_S, \mathbf{h}_E, \mathbf{h}_C), \tag{2}$$

where $\mathcal{F}(\cdot)$ integrates the three feature types in different ways depending on the specific model architecture. In general, after training, the student representation $\mathbf{h}_S$ serves as an interpretable vector reflecting the proficiency of student s_i over the corresponding knowledge concepts. Besides, different CDMs hold different architectures of $\mathcal{F}$, which determines the model's performance and interpretability.

Authorized licensed use limited to: Anhui University. Downloaded on October 26,2026 at 14:55:52 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Existing handcrafted CDMs can be roughly classified into two types regarding their development manners [19]. The first type of CDMs, as well as the most traditional CDMs, is mainly developed by researchers in educational psychology [21], usually utilizing simple functions (e.g., logistic function and the inner-product) as $\mathcal{F}$. Representative CDMs contain item response theory-based models such as IRT [16] and MIRT [37], guess and slipping-based DINA [17], and matrix factorization-based MF [38]. The second type is invented by artificial intelligence researchers, mainly leveraging powerful NNs to enhance the modeling capabilities of $\mathcal{F}$. The representative CDMs conventional theory-based NCD [18], DIRT [39], and KaNCD [40], native concatenation manner-based RCD [41], and novel rule-based KSCD [21]) and PAKP [42]. However, these handcrafted CDMs not only cost design labor and need much trial and error but also cannot cope with complex diagnosis scenarios having larger-scale learning data [23]. Therefore, it is promising and potential to automatically design more effective CDMs via the NAS technique.

B. Evolutionary Neural Architecture Search

As a significant branch of NAS, evolutionary neural architecture search (ENAS) [6] leverages EAs to explore complex search spaces. Compared to other NAS approaches, ENAS excels in easily escaping local optima [43] and achieving trade-offs among multiple objectives through multi-objective optimization [44], enabling the discovery of diverse architectures in a single run. Existing ENAS approaches have been applied in various domains to search different types of neural architectures, including searching convolutional NNs for image classification [45], [46] and object detection [47] in CV area, searching the architectures of recurrent NNs for (NLP) [48] and speech-related [49] tasks, searching graph NNs for non-Euclidean data tasks [50], [51], and searching Transformers for CV [52], NLP [53], speech-related [54] tasks, and so on [55].

Due to the large difference in the search space between different tasks, existing ENAS or NAS approaches cannot be directly applicable to CD [56]. To bridge this gap, Yang et al. suggested a pioneering approach EMO-NAS-CD [23]. It introduces a computation tree-based search space for CD that represents CDMs by trees and employs an MOEA to search high-performance and high-interpretability CDMs by maximizing the objectives of model performance and interpretability. To generate offspring models, four basic tree modifications from GP are empirically combined as a genetic operator for EMO-NAS-CD. After that, another ENAS approach for CD, termed NAS-GCD [22], is developed to search high-generalization CDMs by maximizing model average performance on multiple datasets. NAS-GCD is basically the same as EMO-NAS-GCD but equipped with a solution deduplication strategy to check repeated evaluation models to avoid redundant evaluation. Note that NAS-GCD and EMO-NAS-CD adopt tree-encoding because vector-based encodings suffer from variable-length optimization difficulties and decision constraints [23], which make genetic operators hard to design and often generate infeasible solutions. In contrast, tree-encoding naturally handles architectures of varying complexity and guarantees feasibility, making it a more suitable representation for CDM search.

Despite the promising effectiveness of these ENAS approaches for CD, their convergence speed remains limited due to the simplistic nature of their genetic operators. These operators directly combine basic tree modifications from GP and apply them randomly, without leveraging information about parent models. As noted in [27], [34], this stochastic nature and lack of informed guidance often fail to generate effective offspring models, ultimately slowing convergence. To address these limitations, this paper proposes a novel genetic operator designed to generate more effective CDMs by comprehensively analyzing and leveraging parent model information. Given their exceptional capabilities, LLMs provide a powerful choice for this.

C. Large Language Models

Large language models (LLMs) have rapidly become a central component of modern artificial intelligence due to their large parameter scales and extensive pretraining corpora, which enable them to capture complex linguistic patterns. Representative models such as BERT [57] and GPT [58] have demonstrated strong capabilities in understanding, reasoning, and text generation. These capabilities have recently motivated increasing interest in incorporating LLMs into optimization paradigms, including evolutionary algorithms (EAs) and neural architecture search (NAS) [34]. However, existing studies are largely driven by the characteristics of their target encoding schemes and problem settings.

LLMs for EAs: Existing research that incorporates LLMs into EAs can be broadly categorized into four groups [34]. The first group treats LLMs as evolutionary optimizers, where solution evolution is guided primarily by LLM reasoning, as exemplified by EVOLLM [59] and LMEA [35]. While effective for problems with compact representations, these approaches are not designed to handle structured and constrained representations such as tree-encoded models. The second group employs LLMs as evaluators or surrogate models to approximate objective functions, with LAEA [60] being a representative example. Such methods aim to reduce evaluation cost, but they do not directly address the challenge of generating feasible offspring under complex structural constraints. The third group focuses on using LLMs to emulate mutation and crossover operators, including MOEA/D-LO [61] for real-valued optimization, LMX [62] for text-based solutions, and QDAIF [63] for discrete textual mutation. These methods are tightly coupled with their respective encoding schemes and are not readily applicable to hierarchical or tree-structured representations. The fourth group leverages LLMs to automatically design or generate evolutionary algorithms themselves [64], with representative methods such as AEL [65], EoH [66], LLaMEA [67], and llmEA [68]. Their main idea has been extended to the automatic design of other algorithms [68], [69]. Although effective for algorithm configuration and synthesis, these approaches do not directly consider problem-specific model encoding or feasibility constraints.

Authorized licensed use limited to: Anhui University. Downloaded on October 26,2026 at 14:55:52 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

LLMs for NAS: Related efforts have also explored the use of LLMs in NAS, where existing studies can be categorized according to the role of LLMs in the search process. The first category directly uses LLMs as search strategies to explore architecture spaces, including EvoPrompting [70], GENIUS [71], LLMatic [36], and FL-NAS [72] for CNNs, as well as GHG-NAS [73] and GPT4GNAS [74] for GNNs. These methods primarily focus on fixed-length or graph-based encodings. The second category employs LLMs as performance predictors to accelerate the search process, such as AutoML-GPT [75] and LLM-PP [76]. While effective for reducing evaluation cost, these approaches do not modify the underlying genetic operators. The third category uses LLMs to guide model generation, with GE [77] emulating crossover and mutation over integer-encoded architectures. However, these methods assume relatively simple encoding schemes and do not explicitly address feasibility preservation in variable-length, tree-structured architectures. A small number of studies further explore search space refinement using LLMs [78], but they are not tailored to hierarchical model representations.

Overall, existing LLM-assisted EAs and NAS methods have demonstrated promising results under specific encoding assumptions, including real-valued, text-based, graph-based, or integer-encoded representations. In contrast, leveraging LLMs for *tree-encoded architectures* remains much less explored [79]. Such settings introduce additional challenges, including variable-length optimization, strict feasibility constraints, and hierarchical dependencies among model components. In the context of cognitive diagnosis, these challenges are further amplified by the structured semantics and interpretability requirements of CD models. Motivated by these gaps, this work investigates how LLMs can be effectively integrated as genetic operators for CD-specific tree encoding, rather than directly transferring existing LLM-assisted optimization techniques.

D. Relationship With Transferable NAS

Recent studies on transferable neural architecture search (TNAS) [80], [81], [82] have highlighted the importance of leveraging historical knowledge to accelerate architecture exploration across tasks. For example, EMT-NAS [81] takes the architecture search on each dataset as a task and different tasks finally get different optimal architectures, where multiple tasks exchange knowledge and are solved in an implicit population. Similarly, each task of MTNAS [80] aims to search the optimal architecture on each dataset, which improves NAS efficiency by effective knowledge transfer strategies to to transfer crucial architectural knowledge among multiple tasks. Based on these work, the importance of knowledge transfer mechanisms for evolutionary NAS approaches is highlighted [83].

Our work shares a similar spirit with TNAS in the sense that LLMs serve as a source of prior knowledge to facilitate offspring generation. However, unlike conventional TNAS approaches that focus on the knowledge transfer among multiple tasks, our approach focuses on enhancing the effectiveness of genetic operators within a single-task CD scenario. In this way, our framework complements existing TNAS by extending the concept of transferability from the task level to the operator level, opening a new perspective on how external knowledge sources can be integrated into evolutionary NAS.

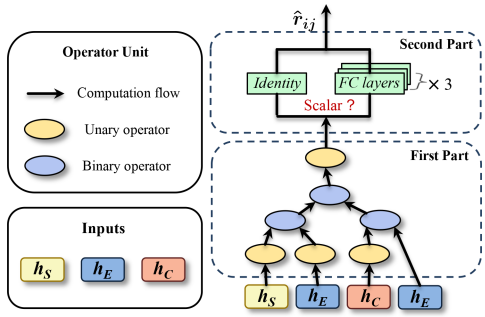


Fig. 1. The tree-based search space for CD: Each model contains several computation nodes and a certain number of input leaf nodes.

III. The Proposed LLM-NAS-CD Approach

This section first gives the adopted search space, encoding, and objectives and then presents the framework of LLM-NAS-CD. Next, the devised LLM-assisted genetic operator is introduced, along with an LLM role-switching strategy. Afterward, the details of the vanilla genetic operator are presented.

A. Search Space, Encoding, & Objectives

1) Search Space: The tree-based search space in EMO-NAS-CD [23] is directly utilized. As shown in Fig. 1, equipped with the tree-based search space, each CDM can be represented by a computation tree followed by an identity operation or three fully connected (FC) layers.

The computation tree has a certain number of leaf nodes, i.e., the inputs selected from $\{\mathbf{h}_S, \mathbf{h}_E, \mathbf{h}_C\}$, and several internal computation nodes, which take at most two inputs and output a scalar or a vector with the size of D through specific operations. There are 15 candidate operations, including nine arithmetic operations (i.e., *Add*, *Mul*, *Neg*, *Abs*, *Inv*, *Square*, *Sqrt*, *Sum*, *Mean*), three non-linear activation-based operations (i.e., *Tanh*, *Sigmoid*, *Softplus*), and three NN mapping-based operations (i.e., *Concat*, *FFN*, *FFN_D*).

As a result, the changeable components in the computation tree lie in: (1) The number of internal computation nodes, which has an upper bound of 20 in our approach; (2) The type of internal computation nodes, where *Concat*, *Add*, and *Mul* are binary operators while others are unary operators; (3) The number of leaf nodes, which is determined by both (1) and (2); (4) The type of leaf nodes, which is one of $\{\mathbf{h}_S, \mathbf{h}_E, \mathbf{h}_C\}$; (5) The specific operator selected from binary operators or unary operators for each internal computation node; (6) The connections between nodes, which are also determined by (1) and (2). Note that, to determine the above changeable components for building a computation tree, the search algorithm only needs to first determine (1) the number of internal computation nodes and (5) the specific operator for each computation node, and then

Authorized licensed use limited to: Anhui University. Downloaded on October 26,2026 at 14:55:52 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Algorithm 1: LLM-NAS-CD Framework.

Input: Pop (Population size), Gen (Maximal number of generations),
Output: P_{non} (Non-dominated solutions)
1 $P \leftarrow$ Randomly initialize Pop solutions;
2 $HV_List \leftarrow \emptyset$; //Store HV values to switch LLM roles
3 **for** $g = 1$ *to* Gen **do**
4 $P' \leftarrow$ MatingPoolSelection(P, N);
5 $P1', P2' \leftarrow P'[1 : \frac{Pop}{2}], P'[\frac{Pop}{2} + 1 : Pop]$; //Evenly divide the mated population
6 $HV_List \leftarrow HV_List \cup$ Hypervolume(P);
7 $P_{HoF} \leftarrow$ Extract top-T solutions from P; //Store better solutions as HoF
8 $role \leftarrow$ LLM role-switching strategy (HV_List,g, Gen); //**Algorithm 3:** Determine the role of LLM according to the HV changes of population
9 $Q1 \leftarrow$ LLM-assist Genetic Operator($role$, P_{HoF},$|P1'|$); //**Algorithm 2:** an LLM-assisted genetic operator by selecting a suitable LLM
10 $Q2 \leftarrow$ Vanilla Genetic Operator($P2'$); //Increase population diversity by introducing the vanilla genetic operator
11 Evaluate each solution in $Q1 \cup Q2$ via Eq.4 after training corresponding models for Num_E epochs;
12 $P \leftarrow$ Environmental Selection$(P \cup Q1 \cup Q2)$;
13 Select non-dominated solutions P_{non} from P;
14 **return** P_{non};

determine (4) the type of leaf nodes, where (2), (3), and (6) are automatically determined.

Based on the above design, a CDM $\mathcal{F}(\cdot)$ in (2) can be rewritten as

$$\mathcal{F}(\mathbf{h}_S, \mathbf{h}_E, \mathbf{h}_C) = \begin{cases} \hat{y} = \mathcal{M}_1(\mathbf{h}_S, \mathbf{h}_E, \mathbf{h}_C) \\ \hat{r}_{ij} = \mathcal{M}_2(\hat{y}) \end{cases}$$

$$\mathcal{M}_2(\cdot) = \begin{cases} Identity(\cdot), \text{if } \hat{y} \in \mathbb{R}^1 \\ FC_3(FC_2(FC_1(\cdot))), \text{if } \hat{y} \in \mathbb{R}^{1\times D} \end{cases}, \tag{3}$$

where $\mathcal{M}_1$ denotes the tree to handle $\mathbf{h}_S$, $\mathbf{h}_E$, and $\mathbf{h}_C$ to output a scalar or a vector, and $\mathcal{M}_2$ is the identity operation or three FC layers.

2) Encoding: Therefore, it is intuitive to adopt the tree representation to encode each model, which can indirectly avoid the variable-length optimization difficulty. For better observation, in Fig. 2, three typical CDMs (NCD, RCD, and MF) are represented by trees. For consistency, both 'trees' and 'solutions' terms are the same in subsequent, denoting the candidate CDMs.

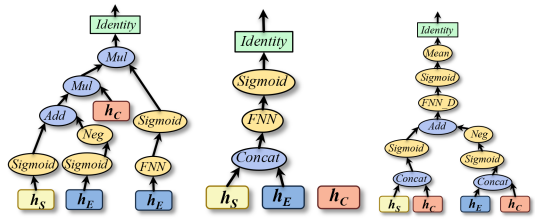
3) Objectives: To search promising CDMs with high performance and interpretability [84], the following bi-objective optimization problem is used:

$$\max_{a\in\mathcal{A}} F(a) = \begin{cases} AUC(a,\ D_{val}|\ D_{train}) \\ \text{model interpretability(a)} \end{cases}, \tag{4}$$

where $\mathcal{A}$ denotes the whole search space. The first objective is the AUC (*Area Under an ROC Curve*) performance of model a on validation dataset D_{val} after training on D_{train} for Num_E epochs, and the second objective adopts the model topology interpretability in EMO-NAS-CD [23]. The model topology interpretability, denoted as $f_{inter}(a)$, is computed as

$$f_{inter}(a) = \left(1 - \frac{depth - 1}{para_1}\right) + \frac{breadth}{para_2} + \left(para_3 - \frac{num_c}{para_4}\right), \tag{5}$$

where $depth$ denotes the tree depth from the lowest-level leaf node to the root node, $breadth$ denotes the tree breadth that can be measured by the number of leaf nodes, and num_c denotes the number of internal computation nodes. According to [23], $(para_1, para_2, para_3, para_4)$ are set to $(10, 200, 0.001, 20000)$. This interpretability objective prefers the trees that have smaller depth, larger breadth, and fewer computation nodes, representing a better model topology interpretability.



(a) NCD. (b) MF. (c) RCD.

Fig. 2. The tree representations of three typical classical CDMs.

B. Overall Framework of LLM-NAS-CD

Algorithm 1 and Fig. 3 illustrate the main steps and the overview of the proposed LLM-NAS-CD approach. The LLM-NAS-CD follows the basic procedure of NSGA-II [24], i.e., continuously from parent selection to offspring generation to offspring evaluation to environmental selection, while its main difference is the adopted LLM-assisted genetic operator along with an LLM role-switching strategy. In this work, the LLM-assisted genetic operator refers to a replacement of the conventional GP-based mutation and crossover operations, where candidate offspring architectures are directly generated by querying the LLM with structured prompts conditioned on parent architectures and the historical archive. To be specific, the algorithm starts with a randomly generated population P and an empty list HV_List to store the hypervolume [85] (HV) value of the population at each generation (Lines 1-2). Second, Pop parent solutions P' are mated through a standard mating pool selection (Line 4), and the mated population P' is evenly divided into two parts $P1'$ and $P2'$ (Line 5), i.e., $P1' = P'[1 : \frac{Pop}{2}]$ and $P2' = P'[\frac{Pop}{2} + 1 : Pop]$. After calculating the HV value of P to update the HV list and keeping the top-T best solutions in P as the Hall of Fame (HoF) solutions P_{HoF} according to SPEA2 [86] (Lines 6-7), an LLM role-switching strategy is used to determine the LLM's role based on the updated HV list (Line 8), and an LLM-assisted genetic operator is used to generate offspring population $Q1$ with a size of $|P1'|$ based on the given

Authorized licensed use limited to: Anhui University. Downloaded on October 26,2026 at 14:55:52 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

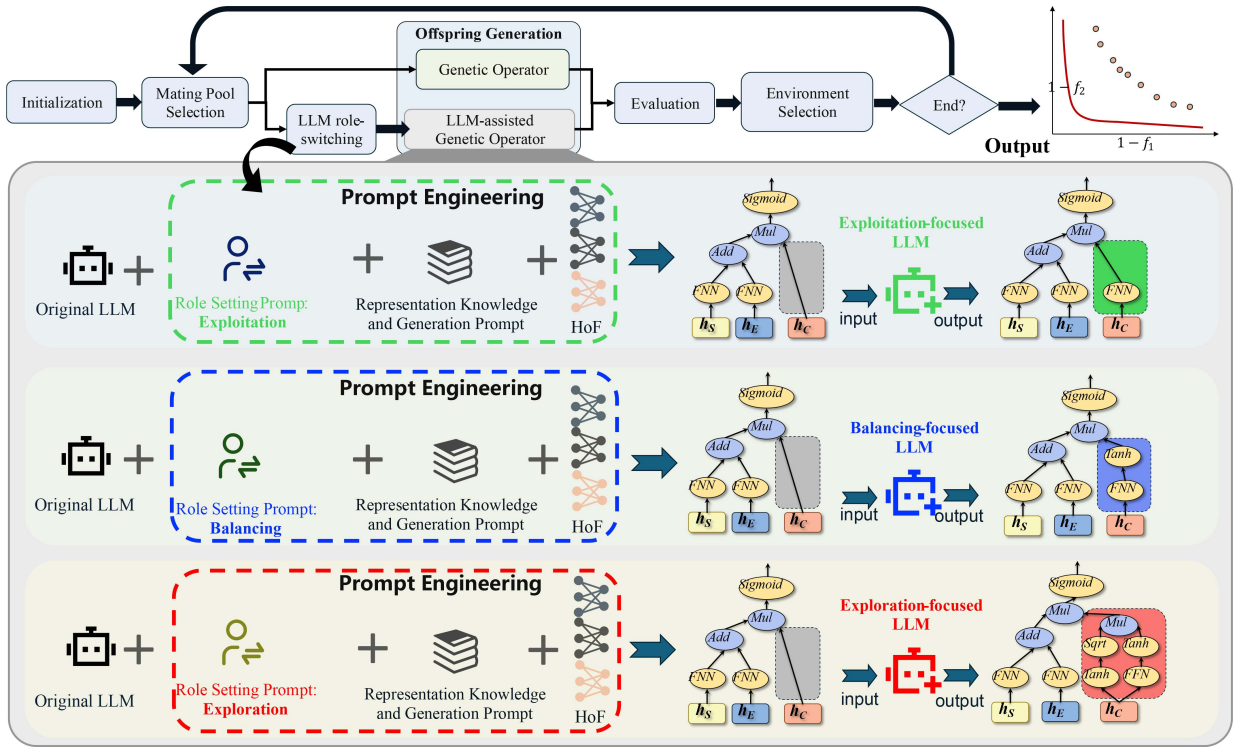


Fig. 3. Overview of the proposed LLM-NAS-CD framework, which follows the basic procedure of EAs but adopts an LLM-assisted genetic operator with the roles of LLM switched by the LLM role-switching strategy. The lower part illustrates the main idea of the devised LLM-assisted genetic operator: three roles of LLMs are defined, including an exploitation-focused LLM, an exploitation-focused LLM, and a balancing-focused LLM, for addressing different scenarios.

$role$ and P_{HoF} (Line 9). Fourth, the mated population $P2'$ is processed by a vanilla genetic operator in EMO-NAS-CD [23] to generate offspring population $Q2$ (Line 10), where the different search behaviors of the vanilla genetic operator can improve the population diversity regarding model architectures. Next, the objective values of each solution in $Q1 \cup Q2$ are obtained according to (4) after training corresponding models for $NumE$ epochs (Line 11). Sixth, better solutions in $P \cup Q1 \cup Q2$ will be kept through the environmental selection of NSGA-II (Line 12). The second to the sixth step will repeat until the maximal number of generations is met, and then non-dominated solutions will be output (Lines 13-14).

C. LLM-Assisted Genetic Operator

The core idea of this operator is to define three roles for LLMs in the offspring generation process to address different scenarios. At each iteration, one appropriate role of LLMs will be used according to the current population state. The three roles of LLMs include an exploitation-focused LLM, an exploration-focused LLM, and a balancing-focused LLM.

To make the created LLMs generate valid and effective solutions, the structured prompt engineering is set to hold four parts, including *a role setting prompt*, *a solution representation knowledge prompt*, *a solution generation prompt*, and *a Hall Of Fame (HoF) prompt*, where the latter three prompts are same for all three roles.

1) The Role Setting Prompt: To start with, it is important for each LLM to specify their roles, making them focus on different search behaviors.

The exploitation-focused LLM aims to make minor adjustments to given architectures in order to search for more complex and high-performance architectures, and thus the following prompt is adopted.

Prompt for the exploitation-focused LLM

```
1 You are an expert in analyzing and optimizing the performance of
      complex neural network architectures. You can deeply understand
      and evaluate the potential of each neural network module,
      focusing on improving network accuracy.
2 When selecting or optimizing network architectures, prioritize
      designing complex network architectures to enhance performance
      by adding minor components to existing excellent architectures.
3 Next, based on the existing architectures provide, design new and
      superior network architectures.
```

Within this prompt, the first sentence instructs the LLM that its basic responsibility is to design and optimize the network architectures, the second further emphasizes its exploitation role to design complex and effective networks by minor incremental adjustments to existing architectures, and the third gives the LLM a command to design new architecture based on existing architectures. Note that the existing architectures will be specified in the subsequent HoF prompt.

Differently, the exploration-focused LLM aims to explore innovative model architectures that are quite different from existing ones through creative combinations, and thus the following prompt is adopted.

Prompt for the exploration-focused LLM

```
1 You are an expert in neural network architecture design with
      innovative thinking. Focus on exploring new network
      architectures that have not been widely used, aiming to break
      the limitations of traditional designs.
2 Your task is to push the boundaries of network architecture design by
      creatively combining new layers, modules, or connection methods
      to seek more promising architectures.
3 Next, read the current excellent architectures and design novel
      architectures different from the current optimal ones.
```

Similarly, the first sentence gives the basic responsibility instruction to the LLM, the sentence emphasizes its exploration

Authorized licensed use limited to: Anhui University. Downloaded on October 26,2026 at 14:55:52 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

role to design some architectures that have not appeared previously by creatively combining new layers, modules, or connections from existing architectures, and the third assigns the LLM an execution command on existing architectures.

The balancing-focused LLM aims to generate architectures that are between architectures generated by the first two LLMs, achieving a harmonious integration of exploitation and exploration. Accordingly, the following prompt is utilized to guide the LLM.

Prompt for the balancing-focused LLM

```
1 You are an expert skilled at balancing innovation and performance in
      network architecture design.
2 You can deeply analyze the potential of complex architectures and
      identify the advantages of innovative designs, building
      optimized solutions by balancing both aspects. In evolutionary
      computation for generating new network architectures, evaluate
      and provide feedback from both innovation and performance
      perspectives.
3 Next, read the current excellent architectures and innovate based on
      the existing ones.
```

Here, the first sentence directly specifies the LLM's role to achieve a balance between innovation architectures and high-performance architectures, the second provides detailed steps, and the third issues an execution command.

2) The Solution Representation Knowledge Prompt: To make the LLMs understand the structures of solutions and the tree topology, the following prompt is adopted.

```
1  Each network architectures is represented as a directed acyclic graph
       in a one-dimensional array, where each node is represented by 3
       integers.
2  For example: [1,0,9,3,0,0,0,0,9,4,5,10,1,0,9,6,7,11,8,0,6], where:
3  - The first two numbers indicate the child node indices, and the
       third number is the operator code;
4  - Unary operators: Only the first number is valid, the second number
       is 0;
5  - Binary operators: Both numbers are valid indices;
6  - Three fixed leaf nodes are represented as: Node0 (student), Node1 (
       exercise), Node2 (knowledge);
7  - The array does not include the default existing leaf nodes;
8  - The first three numbers in the array are Node3, the next three are
       Node4, and so on.
9  - The array stores information excluding the leaf nodes, e.g.,
       [3,2,10] means the left node is Node3 (first three numbers), the
       right node is Node2, and operator code 10 represents 'add'.
10 - To ensure performance, architectures with at least 3 non-leaf nodes
       should be output.
11
12 Operation codes:  0: neg, 1: abs, 2: square, 3: sqrt, 4: tanh,  5:
       inv, 6: sigmoid, 7: softplus, 8: sum, 9: ffn,  10: add, 11: mul,
       12: concat, 13: mean
```

As can be seen, the first line specifies the solution structure knowledge to the LLM, where the information that per three integers denote each node is also fed. To help the LLM further understand this, lines 2 to 10 give an example by explaining the meaning of which bit. Besides, line 12 presents the candidate operation information to the LLM.

To show whether the LLM understands the solution structure from the given integer vector, the following shows an example of requiring the LLM to present the solution structure for a given vector after feeding *the solution representation knowledge prompt*.

INPUTS

```
1 Please analyze the given integer vector to give the formula for the
      corresponding model and plot the model architecture.
2 The given vector:[0,0,2,1,2,11,3,4,10,5,0,7,6,0,4]
```

ANSWERS

```
1  Calculation expression:
2          tanh(softplus((square(student) + (exercise * knowledge))))
3  Here's how the computation tree looks:
4                              Node7 (tanh)
5                                 |
6                              Node6 (softplus)
7                                 |
8                              Node5 (add)
9                              /      \
10                   Node3 (square)  Node4 (mul)
11                     |                /     \
12                  Node0           Node1      Node2
13                (student)     (exercise)    (knowledge)
```

3) The Solution Generation Prompt: After telling the LLM the solution structure knowledge, the following solution generation prompt is needed to require the LLM to generate valid solutions because there are many structural constraints for trees as shown in [23].

```
1 To generate valid architectures, the following requirements must be
      met:
2 1. Each architecture is a directed acyclic graph or tree(with exactly
       one output).
3 2. Unary operators have only one input value (the other is filled
      with 0).
4 3. Each solution is a one-dimensional array, with lengths between 5*3
       and 20*3.
5 4. Each node cannot use nodes that are not defined later. e.g., [3,
      2, 10] is invalid for the first node because nodes 3 and 4 are
      not defined.
```

Here, the above four rules require the LLM to generate valid trees.

4) The Hall of Fame Prompt: As stated above, each LLM has to generate potential model architectures based on existing promising architectures. Here these promising architectures are called *Hall of Fame* (HoF) architectures, which are the top-T best solutions at each generation regarding the fitness computed by SPEA2 [86]. Therefore, the HoF prompt needs to be updated at each generation based on the following prompt template.

```
1 Here are the superior architectures of the current generation for
      reference. Based on your role, analyze and generate solutions.
2                 solution 1: [*,*,*,...,*,*,*];
3                 solution 2: [*,*,*,...,*,*,*];
4                  ..............................
5                 solution T: [*,*,*,...,*,*,*];
```

Here, the information for T HoF solutions is not given and will be updated when used at each generation.

5) The Operator Procedure: Equipped with the above prompts, the LLM-assisted genetic operator is constructed to generate offspring solutions by integrating LLM-based generation with deterministic feasibility control. The overall procedure is detailed in Algorithm 2.

First, the offspring set Q_1 is initialized as empty (Line 1). Given the selected LLM role $role$ and the Hall-of-Fame set P_{HoF}, all tree-encoded architectures in P_{HoF} are converted into their corresponding integer-encoded representations, denoted as P_{HoF}^{inter}, which are required for interaction with the LLM (Line 2). Based on P_{HoF}^{inter}, the HoF prompt is constructed and, together with the role-setting prompt of $role$, the solution representation knowledge prompt, and the solution generation prompt, forms a complete prompt $Prompt$ for the LLM (Line 3-4).

Next, the LLM API is called with the constructed prompt $Prompt$ to generate N_1 candidate offspring solutions P_{temp} in integer-encoded form (Line 6). For each candidate solution $x \in P_{\text{temp}}$, a deterministic validity checking procedure is applied. Specifically, x is first converted into a tree-encoded representation x^{tree}, and is considered valid if and only if: (1) x^{tree} forms a single-output directed acyclic graph (DAG), and (2) all node references correspond to previously defined nodes, i.e., no undefined node references exist (Line 8-9). Only solutions satisfying both conditions are retained and added to the offspring set Q_1 (Line 10).

The above process is repeated until the required number of offspring solutions in Q_1 is reached (Line 5-12). It is worth noting that different LLM roles and dynamically updated Hall-of-Fame solutions lead to distinct search behaviors of the LLM-assisted operator. Moreover, the LLM is invoked online at each generation to generate offspring conditioned on the current

Authorized licensed use limited to: Anhui University. Downloaded on October 26,2026 at 14:55:52 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Algorithm 2: LLM-assisted Genetic Operator.

Input : $role$ (selected LLM role), P_{HoF} (Hall-of-Fame solutions), N_1 (required offspring size)
Output : Q_1 (offspring solutions generated by LM)

1 $Q_1 \leftarrow \emptyset$;
2 $P_{\text{HoF}}^{inter} \leftarrow$ Convert all tree-encoded architectures in P_{HoF} into integer-encoded representations;
3 HoF prompt$\leftarrow$ Construct the HoF prompt using the P_{HoF}^{inter};
4 $Prompt \leftarrow$ Construct the complete prompt by concatenating: (i) role-setting prompt of $role$, (ii) solution representation knowledge prompt, (iii) solution generation prompt, and (iv) HoF prompt;
5 **while** $|Q_1| < N_1$ **do**
6 $P_{\text{temp}} \leftarrow$ Call LLM API to generate $N1$ candidate solutions (integer-encoded) with $Prompt$;
7 **foreach** $x \in P_{temp}$ **do**
`//ValidityCheck: Check the validity of generated solutions and remove the invalid`
8 $x^{tree} \leftarrow$ Convert x into a tree representation;
9 **if** x^{tree} *forms a single-output DAG* ***and*** *contains no undefined node references* **then**
10 $Q_1 \leftarrow Q_1 \cup \{x^{tree}\}$;
11 **if** $|Q_1| \geq N_1$ **then**
12 break;
13 **return** Q_1;

population state, while the combination of structured prompts and deterministic validity checking effectively reduces invalid solutions and keeps the computational overhead negligible. Note that the prompt is constructed once per operator call, since the HoF set is fixed during a single invocation of the LLM-assisted operator.

6) Technical Summary: It is worth emphasizing that our contribution is not a simple replacement of traditional operators with LLMs. Instead, we design an LLM-based operator tailored for tree-encoded CDMs, which handles variable-length architectures, preserves feasibility, and leverages hierarchical prompts to guide offspring generation. Combined with empirical operators, this hybrid design enables both effectiveness and diversity, distinguishing our method from existing LLM-assisted approaches. Furthermore, it is worth noting that the LLM is only employed to generate candidate CDM architectures during the evolutionary process, while the training and evaluation of all models are conducted under the same settings as the baselines.

D. LLM Role-Switching Strategy

Previous research [87], [88] highlights the importance of using different strategies for EAs at various stages. In the early stage, exploration strategies maintain diversity and prevent premature convergence; In the middle stage, balancing strategies refine solutions while exploring nearby areas; While in the final stage, exploitation strategies intensify the search for the best solutions.

To switch the roles of LLMs for rapid convergence, a metric-driven LLM role-switching strategy is devised, whose main idea is to judge which state the population is in according to the HV changes and then determine a suitable LLM role.

For better understanding, Algorithm 3 summarizes its basic steps. Specifically, when there are no sufficient HV values (smaller than L), indicating the population is in the early stage, the exploration role-played LLM is directly output (Lines 1-3). Otherwise, it is essential to understand the relative change in trend and magnitude of HV values for determining the role of LLM (Lines 4-13). However, it is a fact that the HV value of the population increases rapidly in the early stage but grows more gradually in the later stage. To mitigate the influence of the absolute magnitude of HV values, a certain number of the latest HV values HV_W are first extracted from HV_List based on a window size W_{Size} and a window rate W_{Rate} (Lines 4-5), which are calculated according to

$$\begin{aligned} W_{Rate} &= \max\left(0.1, \min\left(1 - \frac{\log(g+1)}{\log(Gen+1)}, 0.5\right)\right) \\ W_{Size} &= \max(10, \lfloor W_{Rate} \times |HV_List| \rfloor) \end{aligned} \quad . \quad (6)$$

Then, the latter half of $HV_W = \{HV_i | 1 \leq i \leq W_{Size}\}$ is extracted as $HV_{Back} = \{HV_i | \frac{W_{Size}}{2} + 1 \leq i \leq W_{Size}\}$ to compute the relative improvement ratio of HV within the current window (Line 6). The relative improvement ratio $ImproveRatio$ can be calculated by

$$ImproveRatio = \frac{HV_W[end] - HV_W[1]}{HV_{Back}[end] - HV_{Back}[1] + 1e^{-10}}. \quad (7)$$

$HV_W[1]$ and $HV_W[end]$ denote the first and the last element in HV_W, which is same for $HV_{Back}[end]$ and $HV_{Back}[1]$. Afterward, if there is a large improvement ratio (larger than $threshold_{upper}$), it indicates the population is in rapid convergence, thereby needing an exploitation-focused LLM (Line 9); If there is nearly no improvement (smaller than $threshold_{lower}$), the population may need the exploration to escape local optima, thereby desiring an exploration-focused LLM (Line 11); Otherwise, a balancing-focused LLM will be output.

E. Other Details

In addition to the devised LLM-assisted genetic operator, there is a vanilla genetic operator in Algorithm 1 to generate more diverse solutions.

The vanilla genetic operator generates offspring solutions by randomly applying one of four basic tree modifications to two parent solutions at each step. Four basic tree modifications include *Exchange* that exchanges sub-trees of two parents, *Delete* that randomly deletes one computation node of the given tree, *Insert* that randomly inserts one computation node whose operation is randomly sampled, and *Replace* that replaces one computation node with a randomly sampled node.

Authorized licensed use limited to: Anhui University. Downloaded on October 26,2026 at 14:55:52 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Algorithm 3: LLM Role-Switching Strategy.

Input : HV_List (HV list), g (Current generation), Gen (Maximal generations)
Output: $role$ (Output role: exploitation, exploration, or balancing)

if $|HV_List| < L$ **then**
  $role \leftarrow$ exploration; `//Insufficient HV data`
  return $role$;
Compute W_{Rate} and W_{Size} using Eq. 6;
$HV_W \leftarrow$ The last W_{Size} elements of HV_List;
$HV_{Back} \leftarrow$ The latter half of HV_W;
$ImproveRatio \leftarrow$ Compute the ratio by Eq. 7;
if $ImproveRatio > threshold_{upper}$ **then**
  $role \leftarrow$ exploitation ; `// Rapid improvement`
else if $ImproveRatio < threshold_{lower}$ **then**
  $role \leftarrow$ exploration ; `// Severe stagnation`
else
  $role \leftarrow$ balancing ; `// Moderate improvement`
return $role$;

TABLE I
STATISTICS OF FIVE ADOPTED DATASETS

Dataset	# Students/Concepts/Exercises	# Logs
ASSISTments09	4,163/123/17,746	324,572
SLP	1,499/34/907	57,244
FrcSub	536/8/20	10,720
ASSISTments12	27,405/265/47,104	1,867,167
JunYi	1,000/39/712	203,945

IV. EXPERIMENTS

A. Experimental Settings

1) Datasets: For a comprehensive comparison, five popular education datasets were utilized in experiments to verify the effectiveness of LLM-NAS-CD, including ASSISTments2009 [89], SLP [90], and FrcSub [91], ASSISTments2012 [92], and JunYi [20]. The statistics of these datasets are presented in Table I, and the following gives their brief descriptions:

- ASSISTments2009: It was collected by the ASSISTments online education system[1], containing nearly 4 K students, nearly 18 K exercises, and more than 300 K answer logs.
- SLP: It was released in 2021 by AICFE[2], containing students' response logs from three years of study in eight subjects.
- FrcSub: It is a classical small-scale dataset containing a total of 20 questions and eight 8 mathematical knowledge concepts.
- ASSISTments2012: It is a mathematics-related dataset collected by ASSISTments in the 2012-2013 school year, containing about 2 M answer logs from about 27 K students.
- JunYi: It is a public dataset collected by the JunYi[3] education platform, containing nearly 200 K answer logs from about 1 K students.

[1]https://new.assistments.org/
[2]https://aicfe.cn/
[3]https://www.junyiacademy.org/

Following the previous approaches experiences [18], the students having fewer than 15 response logs were removed to ensure enough data for diagnosis modeling, thereby ensuring the convincing of the experiments.

Considering the time cost of LLM-NAS-CD on ASSISTments2012 and JunYi, the architecture search process of LLM-NAS-CD was performed on the three relatively smaller datasets, i.e., ASSISTments2009, SLP, and FrcSub, respectively. The transferability validation of the searched architectures was executed on ASSISTments2012 and JunYi.

2) Compared Baselines and Evaluation Metrics: For comprehensive comparisons, seven representative handcrafted CDMs were taken as baselines, including DINA [17], IRT [16], MIRT [37], MF [38], NCD [18], RCD [41], KaNCD [40], and a more recent model KAN2CD-e [93]. Considering that not all datasets meet the requirements of RCD, a variant of RCD termed **RCD-native** was compared. Furthermore, two NAS-CD approaches, EMO-NAS-CD [23] and NAS-GCD [22], were considered as baselines, where their best-found models were used for comparison.

To measure the model performance, AUC, *accuracy* (ACC) [94], and *root mean squared error* (RMSE) [14], were adopted as the evaluation metrics.

3) Parameter Settings:

- *Model Parameter Settings:* The embedding dimension D in each model was set to the concept number (K), and the output dimensions of three FC layers in (3) were set to 512, 256, and 1.
- *Search Process Parameter Settings:* Within the architecture search of LLM-NAS-CD, in each dataset, each student's answer logs were randomly split for training, validation, and testing following the ratio of 70%, 10%, and 20%. To optimize each model, the *Adam* [95] optimizer, along with a learning rate of 0.001 and a batch size of 128, was adopted to minimize the cross-entropy loss [96] for 30 epochs, i.e., $Num_E = 30$. For the LLM-NAS-CD, population size Pop and the maximal number of generations Gen were set to 50 and 100. Besides, the initial window length L was set to 5, the number of HoF solutions T was set to 10, and two thresholds $threshold_{upper}$ and $threshold_{lower}$ were set to 0.8 and 0.2. Note that EMO-NAS-CD and NAS-GCD have the same population size and the same maximal number of generations as the LLM-NAS-CD for fair comparison, where NAS-GCD actually has three times the number of evaluations in LLM-NAS-CD because NAS-GCD executes the search on three datasets.
- *Model Retraining Setting:* For convincing comparisons, multiple data splitting settings were used to divide each dataset into training and test datasets, including the ratios of 80%/20%, 70%/30%, 60%/40%, and 50%/50%. The best-found CDMs by the proposed LLM-NAS-CD were retrained for 50 epochs, and other settings were kept the same as the above.

All compared CDMs hold their settings in their original papers. All experiments were conducted on an NVIDIA RTX 4090 GPU, and the API of GPT-4o was used for LLM-NAS-CD.

Authorized licensed use limited to: Anhui University. Downloaded on October 26,2026 at 14:55:52 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE II
THE PERFORMANCE COMPARISON BETWEEN THE ARCHITECTURES FOUND BY THE PROPOSED LLM-NAS-CD APPROACH AND COMPARISON CDMS ON SLP, FRCSUB, AND ASSISTMENTS09 DATASETS REGARDING ACC, RMSE, AND AUC VALUES, AVERAGED ON TEN RUNS. **THE BEST RESULTS** IN EACH COLUMN WITHIN EACH DATASET ARE IN BOLD.

Train/Test ratio		50%/50%			60%/40%			70%/30%			80%/20%		
Datasets	**Models**	**ACC**	**RMSE**	**AUC**	**ACC**	**RMSE**	**AUC**	**ACC**	**RMSE**	**AUC**	**ACC**	**RMSE**	**AUC**
SLP	DINA	0.6026	0.5205	0.6441	0.5737	0.5345	0.6238	0.6198	0.5069	0.6688	0.5864	0.5216	0.6561
	IRT	0.6052	0.495	0.7052	0.6384	0.4788	0.7347	0.6498	0.4707	0.7337	0.688	0.4543	0.7711
	MIRT	0.7142	0.4953	0.7589	0.7246	0.4844	0.7747	0.7375	0.4759	0.785	0.7369	0.4768	0.7852
	MF	0.7654	0.4198	0.8226	0.7809	0.4077	0.8355	0.7745	0.4036	0.8368	0.7889	0.3954	0.8448
	NCD	0.7551	0.4423	0.8141	0.774	0.4265	0.8332	0.7709	0.4246	0.8332	0.7779	0.4009	0.8402
	RCD-native	0.7871	0.3874	0.8504	0.786	0.3892	0.8529	0.7928	0.3856	0.8569	0.7932	0.3792	0.8599
	KaNCD	0.7623	0.4133	0.8186	0.7765	0.3847	0.8306	0.7745	0.3697	0.8306	0.8001	0.3819	0.8542
	KAN2CD-e	0.7868	0.3926	0.8512	0.7982	0.3811	0.8529	0.7926	0.3808	0.8523	0.8156	0.3675	0.8679
	EMO-NAS-CD	0.7905	0.3838	0.8548	0.7896	0.3873	0.8483	0.795	0.3831	0.8552	0.8006	0.3762	0.8682
	NAS-GCD	0.7928	0.3854	0.8521	0.7886	0.3874	0.8483	0.7962	0.1462	0.8587	0.806	0.3824	0.8631
	LLM-NAS-CD	**0.7932**	**0.3835**	**0.8551**	**0.7914**	**0.3864**	**0.8495**	**0.7994**	**0.3807**	**0.859**	**0.8122**	**0.3666**	**0.8745**
FrcSub	DINA	0.5873	0.4997	0.5987	0.5924	0.5012	0.6537	0.6824	0.4765	0.7362	0.6405	0.4712	0.7962
	IRT	0.5973	0.501	0.6588	0.6023	0.502	0.667	0.7014	0.4535	0.7422	0.6599	0.4802	0.8099
	MIRT	0.7807	0.4398	0.8475	0.8029	0.4232	0.8605	0.7938	0.4218	0.8637	0.8069	0.4199	0.8759
	MF	0.5041	0.5	0.5623	0.4895	0.5004	0.6977	0.4558	0.5016	0.6402	0.6524	0.4998	0.7324
	NCD	0.7518	0.4844	0.829	0.7616	0.4737	0.8293	0.7986	0.4385	0.86	0.7728	0.468	0.8601
	RCD-native	0.8117	0.3717	0.8826	0.8229	0.3652	0.8868	0.8283	0.3634	0.8922	0.8306	0.3508	0.8982
	KaNCD	0.7957	0.3806	0.8706	0.8172	0.3694	0.8806	0.8184	0.364	0.8834	0.8298	0.3667	0.8823
	KAN2CD-e	0.8217	0.3564	0.8881	0.8286	0.3543	0.8898	0.8316	0.3592	0.8931	0.8468	0.3327	0.9117
	EMO-NAS-CD	0.8446	0.3396	0.9132	0.8385	0.3397	0.9137	0.8486	0.3381	0.9143	0.8652	0.3209	0.9228
	NAS-GCD	0.8351	0.3407	0.9128	0.8371	0.3412	0.9118	0.8364	0.3421	0.9108	0.8451	0.3357	0.9153
	LLM-NAS-CD	**0.8541**	**0.3332**	**0.9201**	**0.8578**	**0.3302**	**0.9217**	**0.8555**	**0.3282**	**0.9234**	**0.8741**	**0.3165**	**0.9296**
ASSISTments09	DINA	0.6681	0.4815	0.7133	0.6611	0.4695	0.7231	0.6681	0.4698	0.7263	0.6786	0.4626	0.7364
	IRT	0.6941	0.4586	0.7261	0.7032	0.4531	0.7216	0.7181	0.4432	0.7263	0.7013	0.4435	0.7433
	MIRT	0.6784	0.5303	0.7161	0.681	0.5233	0.7265	0.6846	0.5213	0.7316	0.7165	0.4861	0.7265
	MF	0.6813	0.4645	0.7212	0.6789	0.5162	0.7198	0.6835	0.5135	0.7243	0.6486	0.5035	0.7321
	NCD	0.7164	0.4436	0.7412	0.7213	0.4362	0.7432	0.7282	0.4322	0.7589	0.7236	0.4328	0.76178
	RCD-native	0.7211	0.4268	0.761	0.7236	0.4252	0.7632	0.7289	0.4232	0.7655	0.7314	0.4236	0.7686
	KaNCD	0.7186	0.4333	0.7601	0.7223	0.4341	0.7588	0.7215	0.4324	0.7621	0.7288	0.4321	0.7642
	KAN2CD-e	0.7231	0.4258	0.7602	0.7242	0.4251	0.7609	0.7266	0.4281	0.7622	0.7292	0.4283	0.7632
	EMO-NAS-CD	0.7417	0.4397	0.7797	0.7567	0.4286	0.8009	0.7371	0.4238	0.8189	0.7841	0.4091	0.8407
	NAS-GCD	0.7295	0.4316	0.7648	0.7323	0.4283	0.7694	0.7385	0.4224	0.7784	0.7431	0.4182	0.7849
	LLM-NAS-CD	**0.7464**	**0.4247**	**0.7859**	**0.7599**	**0.4188**	**0.8053**	**0.7722**	**0.4057**	**0.8233**	**0.7884**	**0.3964**	**0.8445**

Our source code will be released at https://github.com/BIMK/Intelligent-Education/tree/main/LLM-NAS-CD.

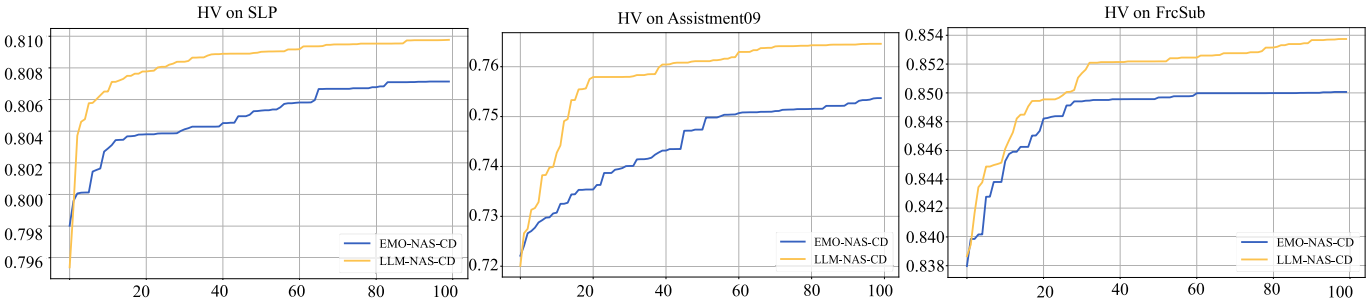
B. Effectiveness of Architectures Found by LLM-NAS-CD

To answer RQ1, Table II compares the performance of seven handcrafted CDMs and CDMs found by EMO-NAS-CD, NAS-GCD, and LLM-NAS-CD on three datasets SLP, FrcSub, and ASSISTments09 regarding the ACC, RMSE, and AUC values. For the results on each dataset, values under four splitting settings were reported, and each value was averaged over ten different runs. Note that there is just one CDM for NAS-GCD on three datasets as NAS-GCD aims to find high-generalization models on multiple datasets; For EMO-NAS-CD, it performed the architecture search on three datasets, respectively, thereby obtaining one CDM for each dataset, where the CDM is the best performance architecture that was selected from multiple Pareto-optimal architectures for comparison; Similarly, there are three CDMs found by LLM-NAS-CD for three datasets, where the best performance architecture was used for comparison.

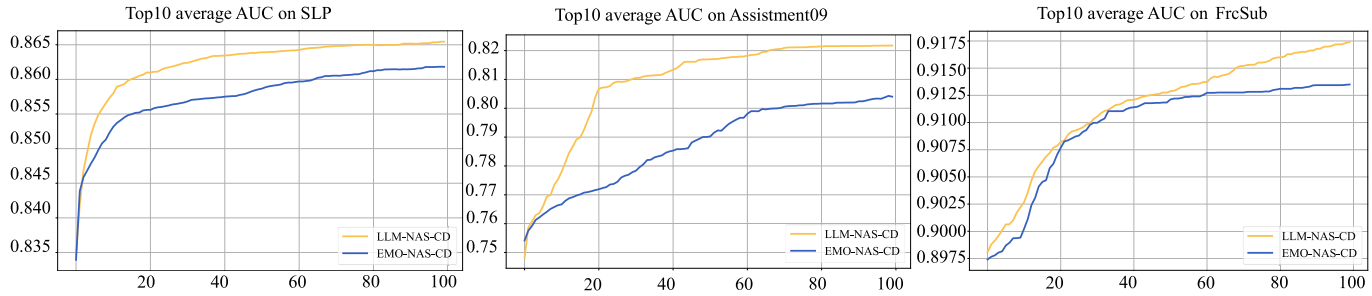
As can be observed from Table II, the CDMs found by the proposed LLM-NAS-CD approach hold better performance than compared CDMs, including handcrafted models and automatically designed models, on all results under three datasets. To be specific, firstly, all NAS-CD approaches, including LLM-NAS-CD, EMO-NAS-GCD, and NAS-GCD, exhibit significantly better performance than handcrafted models, except for the results of NAS-GCD on ASSISTments09 under ratios of 50%/50% and 60%/40% that are just slightly better, where their promising results are mainly attributed to the NAS technique. Even compared to the more recent KAN2CD-e, all NAS-CD approaches still hold better performance. Secondly, the

Authorized licensed use limited to: Anhui University. Downloaded on October 26,2026 at 14:55:52 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.



(a) HV convergence profiles of LLM-NAS-CD and EMO-NAS-CD on three datasets.



(b) Top-ten best architecture average AUC profiles of LLM-NAS-CD and EMO-NAS-CD on three datasets.

Fig. 4. The Comparison between LLM-NAS-CD and EMO-NAS-CD regarding convergence behavior on three datasets. The x-axis denotes the generation index of the evolutionary search, and the y-axis represents the corresponding performance metrics. (a) Hypervolume (HV) convergence profiles on SLP, ASSISTments09, and FrcSub. (b) Top-ten best architecture average AUC convergence profiles on the same datasets.

performance of the best CDMs found by all NAS-CD approaches is easily affected by the amount of training data under the large-scale dataset, but it is the opposite when under small-scale datasets. For example, the AUC performance of LLM-NAS-CD decreases by about 0.06 from the ratio of 80%/20% to the ratio of 50%/50% on the ASSISTments09 dataset but NCD merely decreases by about 0.02, while LLM-NAS-CD just decreases by close to 0.01 from 80%/20% to 50%/50% on FrcSub but NCD decreases by about 0.03. The reason behind this may be that the architectures found by NAS-CD approaches are more complex and thus, more training data may provide models with better capabilities; Thirdly, i.e., most importantly, the performance of the best CDMs found by the LLM-NAS-CD is better than the other two NAS-CD approaches under all results, especially for EMO-NAS-CD because LLM-NAS-CD is the same as EMO-NAS-CD merely except for the search approach design. That indicates the proposed LLM-NAS-CD is more effective than EMO-NAS-CD, attributed to the more novel search approach.

To further investigate the superiority of LLM-NAS-CD to EMO-NAS-CD, Fig. 4(a) plots the HV convergence profiles of LLM-NAS-CD and EMO-NAS-CD obtained on three datasets. Generally, larger HV values represent better convergence and diversity. As can be seen from the comparison on the SLP dataset, LLM-NAS-CD quickly converges to a high level after about the 15th generation, while EMO-NAS-CD converges to the same level after about the 65th generation, needing about an extra 50 generations. Similar observations can be obtained on the other two datasets, but the convergence speed of LLM-NAS-CD on FrcSub is not very significant. The relatively slow speed may be because the FrcSub dataset is too small and promising architectures cannot quickly be distinguished. Furthermore, on the three datasets, the final HV values of LLM-NAS-CD are better than EMO-NAS-CD, where a better final HV value signifies there found more high-performance architectures or more architectures with complex topology by LLM-NAS-CD according to the definition in (4).

To validate this assumption, on the one hand, Fig. 4(b) plots the average AUC profiles of LLM-NAS-CD and EMO-NAS-CD on three datasets, where each AUC value is averaged on the top-ten best performance architectures at each generation. Taking the comparison on the SLP as an example, the average AUC profile of LLM-NAS-CD converges faster and better than that of EMO-NAS-CD, indicating more high-performance architectures are found by LLM-NAS-CD. Similar observations can be obtained on the other two datasets. On the other hand, Fig. 5 presents the architecture visualization of CDMs found by NAS-GCD, EMO-NAS-CD, and LLM-NAS-CD, where CDMs found on SLP, FrcSub, and ASSISTments09 are denoted with the suffix '-S', '-F', and '-A', respectively (except for NAS-GCD). We can find that the CDMs found by LLM-NAS-CD are slightly more complex than those found by EMO-NAS-CD, in terms of model depth and the number of nodes, indicating there are indeed some architectures with more complex topologies found by LLM-NAS-CD. Despite more complex topologies, the interpretability of CDMs found by LLM-NAS-CD is competitive with that of EMO-NAS-CD and that of NAS-GCD. As a result, the above assumption can be validated.

In summary, we can conclude that the proposed LLM-NAS-CD is more effective than EMO-NAS-CD in terms of found model performance and algorithm convergence. That is mainly

Authorized licensed use limited to: Anhui University. Downloaded on October 26,2026 at 14:55:52 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

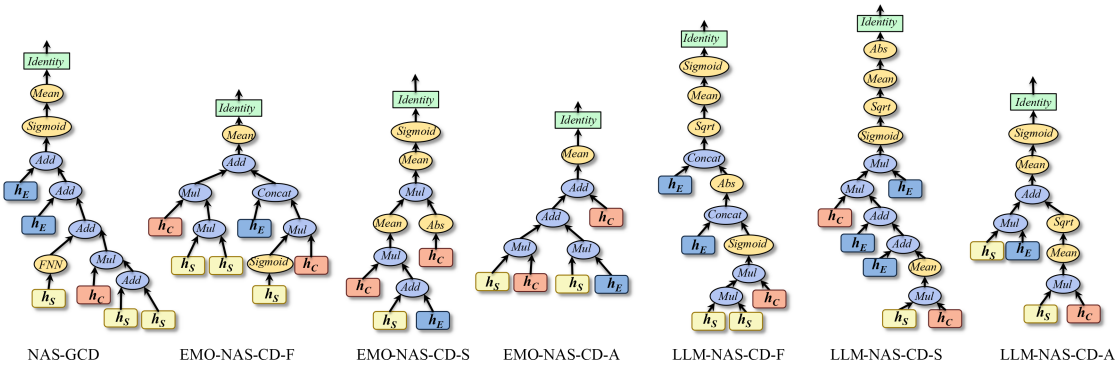


Fig. 5. Architecture visualization of the CDM found by NAS-GCD, three CDMs found by EMO-NAS-CD on three datasets, and three CDMs found by LLM-NAS-CD on three datasets.

TABLE III
PERFORMANCE COMPARISON BETWEEN SEVERAL TYPICAL CDMS AND THE CDMS FOUND BY NAS-GCD, EMO-NAS-CD, AND LLM-NAS-CD ON ON ASSISTMENTS2012 AND JUNYI UNDER THE 80%/20% SPLITTING RATIO, WHERE EACH VALUE IS AVERAGED ON TEN RUNS. **THE BEST RESULTS** IN EACH COLUMN ARE IN BOLD, AND THE SECOND-BEST RESULTS ARE UNDERLINED.

Datasets	ASSISTments2012			JunYi		
Methods	**ACC↑**	**RMSE↓**	**AUC↑**	**ACC↑**	**RMSE↓**	**AUC↑**
MF	0.7390	0.4221	0.7441	0.7662	0.3973	0.8103
NCD	0.7227	0.4313	0.7306	0.7446	0.4169	0.7910
RCD-native	0.7359	0.4225	0.7507	0.7802	0.3989	0.8238
KaNCD	0.7435	0.4536	0.7496	0.7707	0.3968	0.8125
KAN2CD-e	0.7486	0.4322	0.7498	0.7762	0.3872	0.8236
NAS-GCD	**0.7684**	**0.4121**	**0.7564**	0.7814	**0.3882**	**0.8272**
EMO-NAS-CD-S	0.7638	0.4138	0.7529	0.7774	0.3907	0.8239
EMO-NAS-CD-F	0.7649	0.4128	0.7519	0.7748	0.3929	0.8209
EMO-NAS-CD-A	0.7671	0.4236	0.7470	0.7798	0.3913	0.8222
LLM-NAS-CD-S	0.7563	0.4126	0.7532	**0.7939**	0.3896	0.8263
LLM-NAS-CD-F	0.7575	0.4165	0.7459	0.7791	0.3901	0.8248
LLM-NAS-CD-A	0.7631	0.4254	0.7451	0.7767	0.3912	0.8231

attributed to the devised LLM-assisted genetic operator along with the role-switch strategy, whose effectiveness will be validated in subsequent sections.

C. Transferability Effectiveness of Found Architectures

In addition to model performance on the datasets that are observed during the search, it is also important to compare models on some unobserved datasets to evaluate their transferability effectiveness. To this end, Table III compares the performance of several typical CDMs (i.e., MF, NCD, RCD-native, KaNCD, and KAN2CD-e) and the CDMs found by NAS-GCD, EMO-NAS-CD, and LLM-NAS-CD on the other two datasets, ASSISTments2012 and JunYi, under the ratio of 80%/20%. For easy observation, the best results and the second-best results in each column are in bold and underlined, respectively.

As can be seen, NAS-GCD achieves the best performance on nearly all metrics under both two datasets, except for the ACC value on the JunYi dataset. That is reasonable because the aim of NAS-GCD is to search high-generalization CDMs, where its best-found model naturally can hold better transferability than others. Except for NAS-GCD, the CDMs found by EMO-NAS-CD and the CDMs found by LLM-NAS-CD exhibit slightly better performance than handcrafted models. Even compared to the state-of-the-art KAN2CD-e, the CDMs found by LLM-NAS-CD still hold better performance. Among the six searched CDMs, the LLM-NAS-CD-S is the most promising one and is just worse than NAS-GCD, whose performance is better than handcrafted models and the three CDMs found by EMO-NAS-CD. Besides, LLM-NAS-CD-F and LLM-NAS-CD-A also exhibit promising performance.

In summary, the transferability of the CDMs found by LLM-NAS-CD is not better than NAS-GCD but is better than handcrafted models and competitive with EMO-NAS-CD.

D. Effectiveness of Devised Strategies

In RQ1, the whole effectiveness of the devised genetic operator, along with the LLM role-switching strategy, has been validated by comparing LLM-NAS-CD to EMO-NAS-CD. In this subsection, we will comprehensively analyze the effectiveness of the role-switching strategy, the devised prompts, the traditional genetic operator, and the hybrid operator that combines the traditional and LLM-assisted genetic operators. For simplicity and convincing results, the following ablation study will be conducted on the SLP dataset.

To validate the effectiveness of the role-switching strategy, three variants of LLM-NAS-CD were created to use only one role of LLMs during the whole search, including *LLM-NAS-CD(exploit)* using the exploitation-focused LLM, *LLM-NAS-CD(explore)* using the exploration-focused LLM, and *LLM-NAS-CD(balance)* using the balancing-focused LLM. Considering this paper's motivation for using LLM to generate more effective CDMs, Fig. 6 presents the top-ten best-performance architecture average AUC profiles of *LLM-NAS-CD(exploit)*, *LLM-NAS-CD(explore)*, *LLM-NAS-CD(balance)*, and LLM-NAS-CD. Besides, Fig. 6 also compares another variant *LLM-NAS-CD(w/o HoF)*, which is the same as LLM-NAS-CD but does not use the HoF prompt. As can be seen, LLM-NAS-CD holds a faster convergence speed and a better final top-ten AUC value. That indicates that the devised LLM role-switching

Authorized licensed use limited to: Anhui University. Downloaded on October 26,2026 at 14:55:52 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

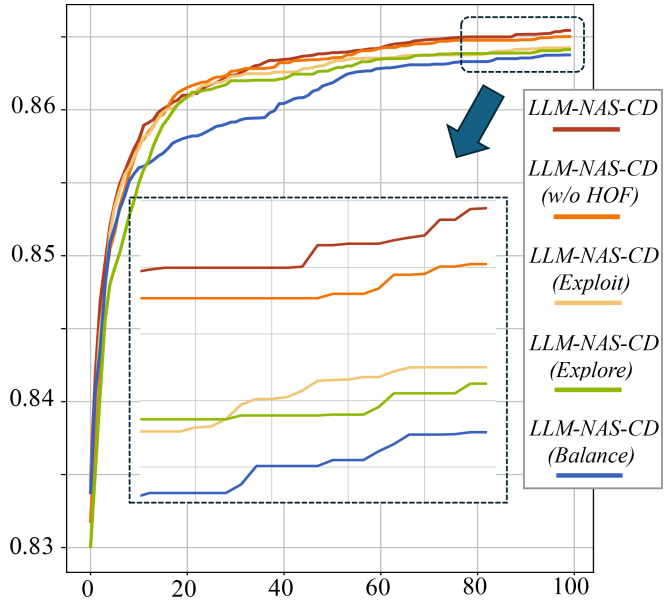


Fig. 6. Top-ten best architecture average AUC profiles of LLM-NAS-CD and its four variants obtained on the SLP dataset.

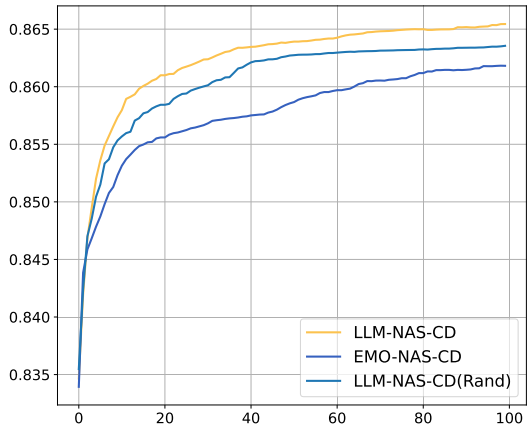


Fig. 7. Top-ten best architecture average AUC profiles of LLM-NAS-CD and random-role-based LLM-NAS-CD (Rand) on the SLP dataset.

strategy and the devised HoF prompt can indeed accelerate and enhance the convergence of LLM-NAS-CD, thereby validating their effectiveness.

To further validate the effectiveness of the role-switching strategy, a variant of LLM-NAS-CD is created as LLM-NAS-CD (Rand), which deletes the 8th row of Algorithm 1 and randomly assigns one role from three candidate roles. Fig. 7 presents the top-ten best-performance architecture average AUC profiles of LLM-NAS-CD (Rand) and LLM-NAS-CD. It is obvious that LLM-NAS-CD (Rand) has a slower convergence speed and a worse final convergence than LLM-NAS-CD. That indicates that the devised role-switching strategy is effective in accelerating and promoting the convergence of LLM-NAS-CD, further validating its effectiveness.

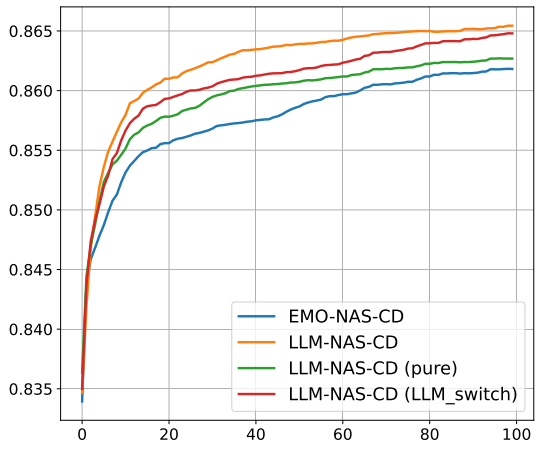


Fig. 8. Top-ten best architecture average AUC profiles of LLM-NAS-CD and other two variants obtained on the SLP dataset.

To validate the effectiveness of the traditional genetic operator and the balancing-focused LLM in the LLM-NAS-CD, two variants of LLM-NAS-CD were created for comparison, termed LLM-NAS-CD (pure) and LLM-NAS-CD (LLM_switch). LLM-NAS-CD (pure) does not utilize the traditional genetic operator by removing the 10th row of Algorithm 1, and LLM-NAS-CD (LLM_switch) does not contain the balancing-focused LLM (i.e., the LLM-based balance genetic operator), where LLM-NAS-CD (LLM_switch) employs another LLM agent to determine using the LLM-based exploitation operator or the exploration operator through replacing the 8th row to the 13th row in Algorithm 3 by feeding the HV list to the LLM agent. Fig. 8 presents the top-ten best-performance architecture average AUC profiles of LLM-NAS-CD (pure), LLM-NAS-CD (LLM_switch) and LLM-NAS-CD. As can be seen, LLM-NAS-CD (LLM_switch) has a faster convergence speed than EMO-NAS-CD but it is still worse than the proposed LLM-NAS-CD from the 20th generation to the end. That further validates the effectiveness of the three different operators designed in the proposed approach. Besides, LLM-NAS-CD (pure) has a significantly slower convergence speed in the early search stage (before the 20th generation), whose final convergence is also significantly worse than LLM-NAS-CD but still better than EMO-NAS-CD. The above observation indicates that it is effective to utilize the traditional genetic operator in the proposed LLM-NAS-CD.

To the effectiveness of the hybrid operator that combines the traditional and LLM-assisted genetic operators, we created two variants of EMO-NAS-CD with with subtree mutation [97] and hoist mutation [98], which are termed EMO-NAS-CD (SubM) and EMO-NAS-CD (Hoist), respectively. Fig. 9 presents the top-ten best-performance architecture average AUC profiles of EMO-NAS-CD, EMO-NAS-CD, EMO-NAS-CD (SubM), EMO-NAS-CD (Hoist), and LLM-NAS-CD. It can be seen that both EMO-NAS-CD (SubM) and EMO-NAS-CD (Hoist) are worse than the proposed LLM-NAS-CD, validating the effectiveness of the overall genetic operator in our approach. Besides,

Authorized licensed use limited to: Anhui University. Downloaded on October 26,2026 at 14:55:52 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

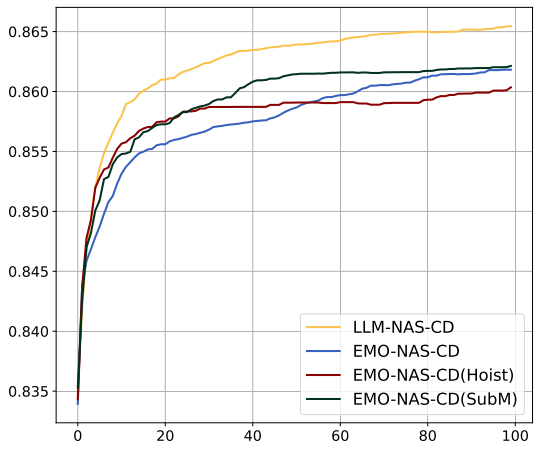


Fig. 9. Top-ten best architecture average AUC profiles of LLM-NAS-CD and two variants of EMO-NAS-CD obtained on the SLP dataset.

TABLE IV
THE RATIOS OF VALID MODELS GENERATED BY THREE ROLES OF LLMS UNDER DIFFERENT PROMPTS

Prompts	Exploitation	Exploration	Balancing
FullPrompt	92.4%	90.4 %	91.2 %
Prompt w/o HoF	49.6 %	47.6 %	47.8 %
Prompt w/o sg	76.6 %	75.2 %	74.2 %
Prompt w/o HoF&sg	36.2 %	36.4 %	36.4 %

we can find that EMO-NAS-CD (Hoist) has a faster convergence speed than EMO-NAS-CD but has a worse final convergence, while EMO-NAS-CD (SubM) is better than EMO-NAS-CD regarding both the convergence speed and final convergence, validating the effectiveness of the hoist mutation. That inspires us to consider more effective mutation operators in future work.

In addition to the influence on the algorithm results, the devised prompts also affect the ratio of valid generated models, which further affects the monetary cost of calling the API because more invalid models mean more times of calling APIs are needed, as shown in line 4 of Algorithm 2. To show the effectiveness of the devised prompts, three roles of LLMs were equipped with four different prompts to generate 5,000 models, respectively. The first prompt is the devised prompts, termed *FullPrompt*, the second is the devised prompts without the HoF prompt, termed *Prompt w/o HoF*, the third is the devised prompts without the solution generation prompt, termed *Prompt w/o sg*, and the fourth is without both prompts, termed *Prompt w/o HoF&sg*.

Table IV summarizes the ratios of the valid models generated by different prompts. As can be seen, the solution generation prompt can improve a ratio of 16% valid models for three roles of LLMs, and the HoF prompt can significantly improve the validity ratio from about 48% to about 91%. The validity ratio will drastically decrease to about 36% when both two prompts are not used, meaning close to twice as many API calls are required. Therefore, the effectiveness of the devised prompts can be demonstrated.

E. Case Study

To observe the usages of three roles of LLMs in the proposed LLM-NAS-CD, Fig. 12 depicts the utilized LLM role at each generation in one single run of LLM-NAS-CD on the SLP dataset: the cyan triangle, the brown inverted triangle, and the green circle represent the exploitation role, the exploration role, and the balancing role, respectively. It can be found that the LLM-NAS-CD employs the exploration role in the early stage after the initial (L=5) generations until the 11th generation, which is consistent with intuition. Then, the exploitation and the balancing roles are continuously switched until about the 42th generation. After close to ten generations of using the exploration role, LLM-NAS-CD again continuously switches the exploitation and the balancing roles until the 85th generation. Here, LLM-NAS-CD is almost converged and thus tries to use the exploration role again until its end, indicating the LLM-NAS-CD is nearly converged. As can be seen, the usage of LLM roles is basically consistent with our original assumption and also consistent with the common optimization process.

For a deep sight into the three roles of LLMs, Fig. 10 gives a toy example to show their behaviors of model generation: an eight-node model (containing five computation nodes) is given as the parent model, and three roles of LLMs are requested to generate an effective more according to the parent. As can be observed, the model generated by exploitation-focused LLM is very similar to the given parent, where one computation node '*Add*' and one leaf node $\mathbf{h}_S$ are added; Differently, the model generated by the exploration-focused LLM holds a huge difference from the parent (only their left parts are same), whose right part seems to be a potential part summarized from experiences; Naturally, the model generated by the balancing-focused LLM strikes a balance between the above two, which is quite close to the parent but has four newly added nodes. In summary, the behaviors of the three roles of LLMs are consistent with their design intention, indirectly demonstrating their effectiveness.

To demonstrate the superiority of the found architectures, we conducted a case study on the SLP dataset. Fig. 11 compares the t-SNE visualizations of knowledge concept embeddings learned by NCD and by LLM-NAS-CD-S. As shown in the figure, although both models capture the underlying structure of the 34 concepts, the embeddings produced by LLM-NAS-CD-S are more clearly separated into five coherent groups, while those learned by NCD remain relatively mixed. This observation highlights that the proposed approach is able to discover models that learn more discriminative and interpretable concept representations.

F. Theory-Linked Diagnosis Analysis Via IRT Consistency

For CD models, it is important not only to achieve strong predictive performance, but also to provide diagnostically meaningful outputs that align with established educational theories. To further evaluate the pedagogical plausibility of the architectures discovered by LLM-NAS-CD, we followed previous research [40] to conduct an analysis inspired by Item Response Theory (IRT) [16].

Authorized licensed use limited to: Anhui University. Downloaded on October 26,2026 at 14:55:52 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

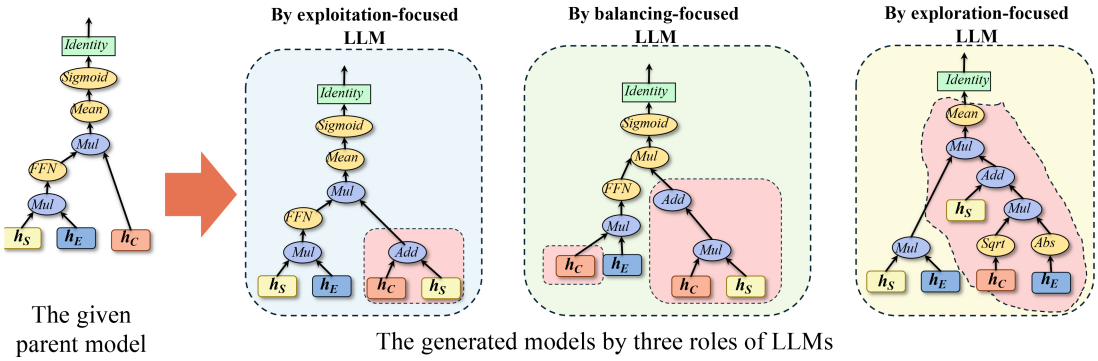


Fig. 10. The model generation behavior comparison among the exploitation-focused LLM, the balancing-focused LLM, and the exploration-focused LLM on a given model.

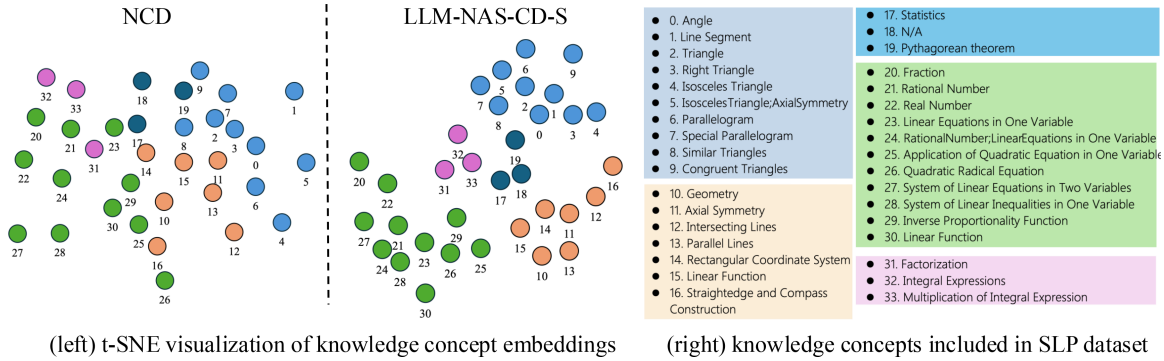


Fig. 11. t-SNE visualization of knowledge concept embeddings learned by the architecture LLM-NAS-CD-S and NCD on the SLP dataset.

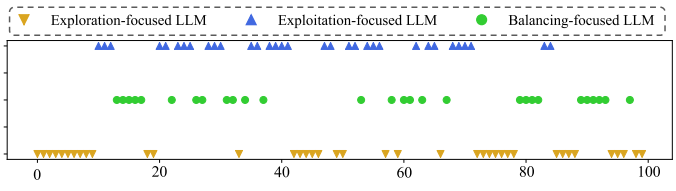


Fig. 12. The role used at each generation in one single run of LLM-NAS-CD on the SLP dataset.

1) Motivation From IRT: A fundamental intuition in IRT is that a student is more likely to answer an exercise correctly when the student's proficiency on the required knowledge concepts exceeds the corresponding exercise difficulty. In the context of cognitive diagnosis, this suggests that the diagnosed proficiency-difficulty relationship should exhibit a monotonic consistency with the predicted responses.

2) Illustrative Case Study: We first present an illustrative example on the ASSISTments09 dataset. Fig. 13(a, left) shows the Q-matrix entries of three selected exercises and the observed responses of a student. Using the same student and exercises, Fig. 13(b) visualizes the diagnosed concept proficiencies and the estimated concept difficulties produced by KaNCD, NCD, and LLM-NAS-CD, respectively. As can be observed, the proficiency-difficulty relationships inferred by LLM-NAS-CD are more consistent with the observed response patterns, compared to the baseline models.

3) Dataset-Level IRT-Consistency Evaluation: To provide a quantitative evaluation, we further assess IRT-consistency on the ASSISTments09 test set, which contains 51,985 interaction logs. For each interaction log (i, j), let $r_{ij} \in \{0, 1\}$ denote the model-predicted response. Let h_{ij}^S denote the diagnosed student proficiency on the target concept(s) required by exercise j, and let h_j^E denote the corresponding exercise difficulty. An interaction is counted as IRT-consistent if it satisfies

$$\left(h_{ij}^S < h_j^E \wedge r_{ij} = 0\right) \text{ or } \left(h_{ij}^S > h_j^E \wedge r_{ij} = 1\right). \quad (8)$$

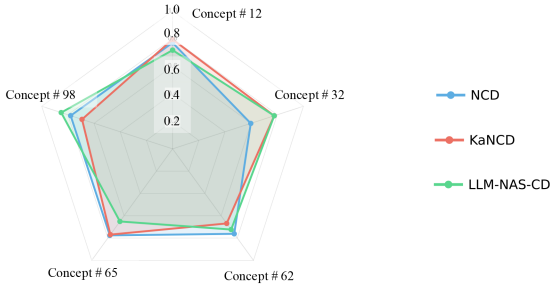
Fig. 13(a, right) reports the overall IRT-consistency ratios of different models. It is unexcepted that LLM-NAS-CD achieves an IRT-consistency ratio of 52.31%, which is substantially higher than those of NCD (34.11%) and KaNCD (28.09%). This indicates that the architectures discovered by LLM-NAS-CD yield diagnostic outputs that are more coherent with the IRT principle.

4) Discussion: This IRT-consistency analysis provides a theory-linked perspective on the interpretability and pedagogical plausibility of the discovered models. While it does not directly measure learning outcomes or intervention effectiveness, it demonstrates that LLM-NAS-CD produces more

Authorized licensed use limited to: Anhui University. Downloaded on October 26,2026 at 14:55:52 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

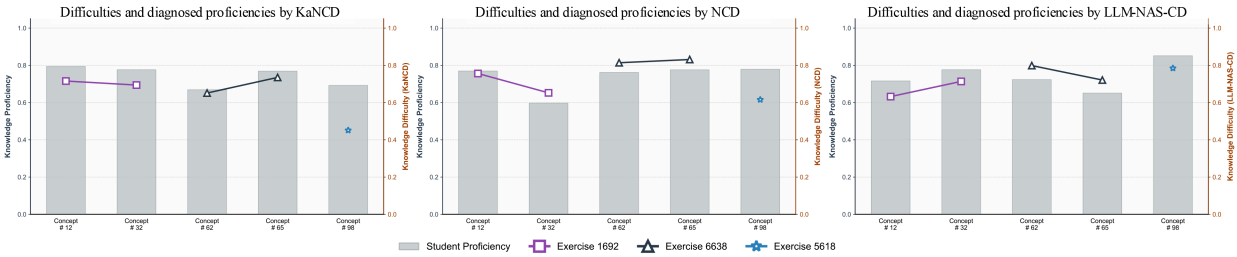
	Concept # 12	Concept # 32	Concept # 62	Concept # 65	Concept # 98	Student Response
Exercise # 1692	1	1	0	0	0	✓
Exercise # 6638	0	0	1	1	0	✗
Exercise # 5618	0	0	0	0	1	✓



A comparison of whether each cognitive diagnosis model follows the IRT theory On the ASSISTments09 dataset

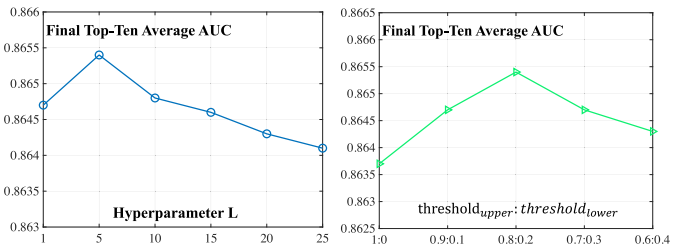
	# Total logs	# Consistent logs	Percentage
KaNCD	51985	14601	28.09%
NCD	51985	17731	34.11%
LLM-NAS-CD	51985	27192	52.31%

(a) **Left**: An illustrative example on ASSISTments09: the Q-matrix entries of three exercises (selected concepts) and the observed student responses. **Right**: IRT-consistency comparison on the ASSISTments09 test set.



(b) Case-study comparison of diagnosed proficiencies and estimated difficulties produced by KaNCD, NCD, and LLM-NAS-CD on the same student and selected concepts on the ASSISTments09.

Fig. 13. The IRT-consistency comparison of NCD, KaNCD, and LLM-NAS-CD on the ASSISTments09 test set.



(a) Hyperparameter L (b) $threshold_{upper}$:$threshold_{lower}$

Fig. 14. Sensitivity analysis of LLM-NAS-CD to hyperparameters L, $threshold_{upper}$, and $threshold_{lower}$ by showing final top-ten average AUC values on the SLP dataset.

coherent proficiency–difficulty relationships, which is desirable for meaningful cognitive diagnosis. Developing intervention recommendation mechanisms and validating them with real educational outcomes will be explored in future work.

G. Hyperparameter Sensitivity Analysis

It is worth noting that there are two sets of hyperparameters in the LLM role-switch strategy, i.e., the initial window length L and the threshold $threshold_{upper}$:$threshold_{lower}$, which may affect the results of LLM-NAS-CD.

To analyze the sensitivity of the proposed LLM-NAS-CD to them, Fig. 14(a) presents the final top-ten average AUC values of LLM-NAS-CD with five different values, i.e., [0,5,10,15,20,25], and Fig. 14(b) depicts the algorithm's final top-ten average AUC values under five different threshold settings, where $threshold_{upper}$ and $threshold_{lower}$ are set to 1:0, 0.9:0.1, 0.8:0.2, 0.7:0.3, and 0.6:0.4, respectively. As can be seen, a smaller value of L, i.e., $L = 0$, leads to poor performance of LLM-NAS-CD due to the insufficient exploration of the algorithm, while much more explorations (i.e., $L \geq 10$) will also cause worse performance. As a result, it is reasonable to adopt $L = 5$ for LLM-NAS-CD. Similarly, the threshold $threshold_{upper}$:$threshold_{lower}$ set to 0.8:0.2 is suitable. Therefore, current parameter settings for LLM-NAS-CD are good enough to some extent.

In addition, we investigated the influence of the *percentage of offspring solutions generated by the LLM-assisted genetic operator*, denoted as $p \in \{0, 25, 50, 75, 100\}$. We denote a variant as **LLM-NAS-CD (mix-p)**, where $p\%$ of offspring are generated by the LLM-assisted operator and the remaining $(100 - p)\%$ are generated by the conventional GP operator. In particular, LLM-NAS-CD (mix-0) is exactly equivalent to EMO-NAS-CD, LLM-NAS-CD (mix-50) corresponds to the proposed LLM-NAS-CD used in the main experiments, and LLM-NAS-CD (mix-100) corresponds to LLM-NAS-CD (pure). Fig. 15 presents the average AUC profiles of the top-ten best architectures for these variants on the SLP dataset. As shown in Fig. 15, LLM-NAS-CD (pure) consistently outperforms EMO-NAS-CD, demonstrating that the LLM-assisted operator itself

Authorized licensed use limited to: Anhui University. Downloaded on October 26,2026 at 14:55:52 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

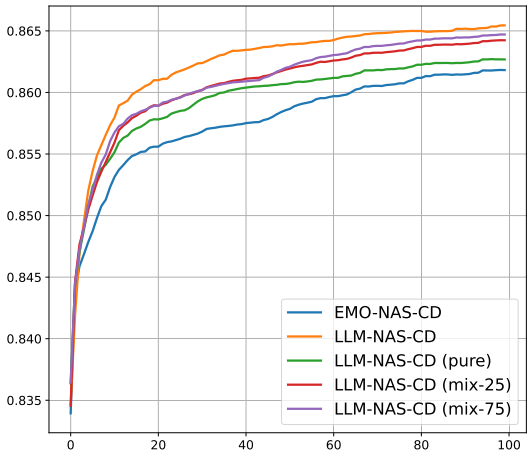


Fig. 15. Top-ten best architecture average AUC profiles of LLM-NAS-CD and its four variants obtained on the SLP dataset, where EMO-NAS-CD, LLM-NAS-CD, and LLM-NAS-CD (pure) are exactly equivalent to LLM-NAS-CD (mix-0), LLM-NAS-CD (mix-50), and LLM-NAS-CD (mix-100), respectively.

TABLE V
COMPARISON OF TIME CONSUMPTION (IN HOURS) BETWEEN LLM-NAS-CD, EMO-NAS-CD, AND NAS-GCD

Dataset	EMO-NAS-CD	LLM-NAS-CD	ΔTime (h/%)	NAS-GCD
SLP	11.5	11.7	+0.2h/+1.7%	401.5
FrcSub	9.3	9.6	+0.3h/+3.2%	
ASSISTments09	112.9	113.3	+0.4h/+0.35%	

is effective in generating more promising offspring solutions by exploiting information from parent architectures. Furthermore, the mixed-ratio variants generally achieve faster convergence and better final performance than either extreme case, indicating that combining LLM-assisted and traditional GP operators helps balance exploitation and exploration during the search. Among them, LLM-NAS-CD (mix-50), i.e., LLM-NAS-CD, achieves the best overall performance, which is consistent with previous studies in evolutionary computation [85].

H. Time Consumption Analysis

One potential concern of incorporating LLMs into the ENAS framework is the additional computational overhead caused by frequent API calls. To quantitatively evaluate this, we compared the search time of our proposed LLM-NAS-CD with EMO-NAS-CD in which LLMs are not involved but the same number of evaluations is used. The runtime of NAS-GCD is also listed. The results are summarized in Table V.

As shown in Table V, the runtime of LLM-NAS-CD is slightly higher than that of EMO-NAS-CD, with a marginal overhead of only 0.2–0.4 hours. This difference accounts for less than 0.5% of the total search time, indicating that the cost introduced by LLM queries is negligible compared to the overall ENAS process. Importantly, this minor increase in runtime is well compensated by the benefits of our method: LLM-NAS-CD significantly improves the effectiveness of offspring generation, accelerates the convergence, and achieves consistently better predictive performance. In addition, the runtime of LLM-NAS-CD is only slightly higher than that of EMO-NAS-CD (e.g., +0.3 h on FrcSub, smaller than 4%). However, Fig. 4 demonstrates that LLM-NAS-CD reaches higher HV and top-ten average AUC much earlier (e.g., generation 80 vs. 100), indicating that our method achieves faster convergence under nearly identical time budgets. Therefore, we argue that the time consumption of LLM-NAS-CD remains practically acceptable, and the trade-off between efficiency and effectiveness is favorable.

In addition to EMO-NAS-CD, the runtime of NAS-GCD is also listed in Table V, which takes about three times that of EMO-NAS-CD as well as LLM-NAS-CD, totally 401.5 hours. It is much expensive because NAS-GCD has to evaluate three objectives on three datasets for each model architecture. This is why we do not validate the LLM-based genetic operator under the NAS-GCD framework. Despite that, we would like to explore a more efficient version of NAS-GCD to validate the LLM-based genetic operator.

V. CONCLUSION AND FUTURE WORK

This paper suggested a novel evolutionary neural architecture search approach for cognitive diagnosis, where an LLM-assisted genetic operator is devised for effective offspring model generation. In the devised genetic operator, three roles of LLMs are defined to address different scenarios through structured prompt engineering. The first LLM focuses on the exploitation of model generation, the second LLM aims to explore more innovative models that have never appeared, while the third balancing-focused LLM aims at balancing exploration and exploitation. To achieve the adaptive role switch of LLMs, an LLM role-switching strategy is devised to determine the population state for selecting LLM roles by observing the population improvement. Besides, the traditional vanilla genetic operator is also utilized to generate partial models for enhancing population diversity. Extensive experiments conducted on five datasets validate the effectiveness of both models found by the proposed approach and the devised strategies.

Despite the promising performance of CDMs found by LLM-NAS-CD and the high effectiveness of the proposed LLM-NAS-CD approach, there still remains one issue of prohibitive computational search cost, e.g., taking about more than ten GPU days on the ASSISTments09 dataset. This issue will become more pronounced when dealing with larger datasets. In the future, we aim to explore surrogate models [99] to help reduce the search costs. By implementing this strategy, we can identify high-performance and high-interpretability CDMs within a more reasonable timeframe. Besides, in this work we built upon EMO-NAS-CD to validate the proposed LLM-assisted operator, mainly for computational feasibility. Since NAS-GCD requires multi-dataset evaluations and thus incurs prohibitive cost, we leave the integration of our operator into the NAS-GCD framework as important future work to further enhance model transferability.

Authorized licensed use limited to: Anhui University. Downloaded on October 26,2026 at 14:55:52 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

REFERENCES

[1] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," in *Proc. 5th Int. Conf. Learn. Representations*, Toulon, France, Apr. 2017, pp. 1–17. [Online]. Available: https://openreview.net/forum?id=r1Ue8Hcxg

[2] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning transferable architectures for scalable image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 8697–8710.

[3] T. Elsken, J. H. Metzen, and F. Hutter, "Efficient multi-objective neural architecture search via Lamarckian evolution," in *Proc. 7th Int. Conf. Learn. Representations*, New Orleans, LA, USA, May 2019, pp. 1–23. [Online]. Available: https://dblp.org/rec/conf/iclr/ElskenMH19.bib

[4] Y. Jaafra, J. L. Laurent, A. Deruyver, and M. S. Naceur, "Reinforcement learning for neural architecture search: A review," *Image Vis. Comput.*, vol. 89, pp. 57–66, 2019.

[5] Y. Akimoto, S. Shirakawa, N. Yoshinari, K. Uchida, S. Saito, and K. Nishida, "Adaptive stochastic natural gradient method for one-shot neural architecture search," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 171–180.

[6] Y. Liu, Y. Sun, B. Xue, M. Zhang, G. G. Yen, and K. C. Tan, "A survey on evolutionary neural architecture search," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 2, pp. 550–570, Feb. 2023.

[7] Y. Fan, F. Tian, Y. Xia, T. Qin, X.-Y. Li, and T.-Y. Liu, "Searching better architectures for neural machine translation," *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 28, pp. 1574–1585, 2020.

[8] N. Wang et al., "NAS-FCOS: Fast neural architecture search for object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 11940–11948.

[9] L. Jiang, Y. Hao, and J. Lin, "Dialogue text summarization method combined self-supervised learning and neural architecture search," in *Proc. Int. Conf. Big Data, Artif. Intell. Risk Manage.*, 2022, pp. 65–69.

[10] A. Mehrotra et al., "NAS-Bench-ASR: Reproducible neural architecture search for speech recognition," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–15.

[11] C. Liu et al., "Auto-DeepLab: Hierarchical neural architecture search for semantic image segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 82–92.

[12] P. Trirat and J.-G. Lee, "PASTA: Neural architecture search for anomaly detection in multivariate time series," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 9, no. 4, pp. 2924–2939, Aug. 2025.

[13] A. Anderson, D. Huttenlocher, J. Kleinberg, and J. Leskovec, "Engaging with massive online courses," in *Proc. Int. Conf. World Wide Web*, 2014, pp. 687–698.

[14] S. Yang et al., "Cognitive diagnosis-based personalized exercise group assembly via a multi-objective evolutionary algorithm," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 7, no. 3, pp. 829–844, Jun. 2023.

[15] Y. Sun, Y. Cao, Z. Wang, S. Hou, W. Gao, and Z.-H. Zhan, "Personalized exercise group assembly using a two archive evolutionary algorithm," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 9, no. 3, pp. 2572–2583, Jun. 2025.

[16] S. E. Embretson and S. P. Reise, *Item Response Theory*. East Sussex, U.K.: Psychol. Press, 2013.

[17] J. D. L. Torre, "Dina model and parameter estimation: A didactic," *J. Educ. Behav. Statist.*, vol. 34, no. 1, pp. 115–130, 2009.

[18] F. Wang et al., "Neural cognitive diagnosis for intelligent education systems," in *Proc. AAAI Conf. Artif. Intell.*, 2020, vol. 34, no. 04, pp. 6153–6161.

[19] F. Wang et al., "A survey of models for cognitive diagnosis: New developments and future directions," 2024, *arXiv:2407.05458*.

[20] H. Ma, C. Wang, H. Zhu, S. Yang, X. Zhang, and X. Zhang, "Enhancing cognitive diagnosis using un-interacted exercises: A collaboration-aware mixed sampling approach," in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 8, pp. 8877–8885.

[21] H. Ma et al., "Knowledge-sensed cognitive diagnosis for intelligent education platforms," in *Proc. 31st ACM Int. Conf. Inf. Knowl. Manage.*, 2022, pp. 1451–1460.

[22] S. Yang et al., "Evolutionary multi-objective neural architecture search for generalized cognitive diagnosis models," in *Proc. Int. Conf. Data-Driven Optim. Complex Syst.*, 2023, pp. 1–10.

[23] S. Yang et al., "An evolutionary multi-objective neural architecture search approach to advancing cognitive diagnosis in intelligent education," *IEEE Trans. Evol. Comput.*, vol. 29, no. 6, pp. 2431–2445, Dec. 2025.

[24] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, Apr. 2002.

[25] Y. Bi, B. Xue, and M. Zhang, "Genetic programming with image-related operators and a flexible program structure for feature learning in image classification," *IEEE Trans. Evol. Comput.*, vol. 25, no. 1, pp. 87–101, Feb. 2021.

[26] Y. Bi, B. Xue, and M. Zhang, "Dual-tree genetic programming for few-shot image classification," *IEEE Trans. Evol. Comput.*, vol. 26, no. 3, pp. 555–569, Jun. 2022.

[27] Z. Wang, S. Liu, J. Chen, and K. C. Tan, "Large language model-aided evolutionary search for constrained multiobjective optimization," in *Proc. Int. Conf. Intell. Comput.*, 2024, pp. 218–230.

[28] A. J. Thirunavukarasu, D. S. J. Ting, K. Elangovan, L. Gutierrez, T. F. Tan, and D. S. W. Ting, "Large language models in medicine," *Nature Med.*, vol. 29, no. 8, pp. 1930–1940, 2023.

[29] W. X. Zhao et al., "A survey of large language models," 2023, *arXiv:2303.18223*.

[30] V. D. Lai et al., "ChatGPT beyond english: Towards a comprehensive evaluation of large language models in multilingual learning," in *Proc. Findings Assoc. Comput. Linguistics*, 2023, pp. 13171–13189.

[31] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, vol. 36, pp. 21558–21572.

[32] B. Romera-Paredes et al., "Mathematical discoveries from program search with large language models," *Nature*, vol. 625, no. 7995, pp. 468–475, 2024.

[33] S. Li et al., "Pre-trained language models for interactive decision-making," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 31199–31212.

[34] X. Wu, S.-h. Wu, J. Wu, L. Feng, and K. C. Tan, "Evolutionary computation in the era of large language model: Survey and roadmap," *IEEE Trans. Evol. Comput.*, vol. 29, no. 2, pp. 534–554, Apr. 2025.

[35] S. Liu, C. Chen, X. Qu, K. Tang, and Y.-S. Ong, "Large language models as evolutionary optimizers," in *Proc. IEEE Congr. Evol. Computation*, 2024, pp. 1–8.

[36] M. U. Nasir, S. Earle, J. Togelius, S. James, and C. Cleghorn, "LLMatic: Neural architecture search via large language models and quality diversity optimization," in *Proc. Genet. Evol. Computation Conf.*, 2024, pp. 1110–1118.

[37] M. D. Reckase, "Multidimensional item response theory models," in *Multidimensional Item Response Theory*. Berlin, Germany: Springer, 2009, pp. 79–112.

[38] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 42, no. 8, pp. 30–37, Aug. 2009.

[39] S. Cheng et al., "DIRT: Deep learning enhanced item response theory for cognitive diagnosis," in *Proc. ACM Int. Conf. Inf. Knowl. Manage.*, 2019, pp. 2397–2400.

[40] F. Wang et al., "NeuralCD: A general framework for cognitive diagnosis," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 8, pp. 8312–8327, Aug. 2023.

[41] W. Gao et al., "RCD: Relation map driven cognitive diagnosis for intelligent education systems," in *Proc. Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2021, pp. 501–510.

[42] H. Ma et al., "A prerequisite attention model for knowledge proficiency diagnosis of students," in *Proc. 31st ACM Int. Conf. Inf. Knowl. Manage.*, 2022, pp. 4304–4308.

[43] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Regularized evolution for image classifier architecture search," in *Proc. AAAI Conf. Artif. Intell.*, 2019, vol. 33, no. 01, pp. 4780–4789.

[44] Z. Lu et al., "NSGA-Net: Neural architecture search using multi-objective genetic algorithm," in *Proc. Genet. Evol. Computation Conf.*, 2019, pp. 419–427.

[45] M. Suganuma, S. Shirakawa, and T. Nagao, "A genetic programming approach to designing convolutional neural network architectures," in *Proc. Genet. Evol. Computation Conf.*, 2017, pp. 497–504.

[46] S. Deng, Z. Lv, E. Galván, and Y. Sun, "Evolutionary neural architecture search for facial expression recognition," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 7, no. 5, pp. 1405–1419, Oct. 2023.

[47] Y. Chen et al., "RENAS: Reinforced evolutionary neural architecture search," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 4782–4791.

[48] K. Kadamala and J. Griffith, "An insight into neuroevolution and genetic algorithms for text classification," *Procedia Comput. Sci.*, vol. 225, pp. 1379–1387, 2023.

Authorized licensed use limited to: Anhui University. Downloaded on October 26,2026 at 14:55:52 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

[49] G. Hickok, J. Venezia, and A. Teghipco, "Beyond Broca: Neural architecture and evolution of a dual motor speech coordination system," *Brain*, vol. 146, no. 5, pp. 1775–1790, 2023.

[50] M. Shi et al., "Genetic-GNN: Evolutionary architecture search for graph neural networks," *Knowl.-Based Syst.*, vol. 247, 2022, Art. no. 108752.

[51] Y. Huang, C. Zhang, and J. Wang, "GNN-EA: Graph neural network with evolutionary algorithm," in *Proc. IEEE Int. Conf. Syst., Man, Cybern.*, 2022, pp. 1476–1481.

[52] S. Yang, X. Sun, K. Xu, Y. Liu, Y. Tian, and X. Zhang, "Hybrid architecture-based evolutionary robust neural architecture search," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 8, no. 4, pp. 2919–2934, Aug. 2024.

[53] Q. Zhou et al., "Training-free transformer architecture search with zero-cost proxy guided evolution," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 10, pp. 6525–6541, Oct. 2024.

[54] X. Shi, P. Zhou, W. Chen, and L. Xie, "Darts-Conformer: Towards efficient gradient-based neural architecture search for end-to-end ASR," 2021, *arXiv:2104.02868*.

[55] R. Lange, Y. Tian, and Y. Tang, "Evolution transformer: In-context evolutionary optimization," in *Proc. Genet. Evol. Computation Conf. Companion*, 2024, pp. 575–578.

[56] S. Yang et al., "Designing novel cognitive diagnosis models via evolutionary multi-objective neural architecture search," 2023, *arXiv:2307.04429*.

[57] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, 2019, pp. 4171–4186.

[58] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," OpenAI preprint, 2018. [Online]. Available: https://www.mikecaptain.com/resources/pdf/GPT-1.pdf

[59] R. Lange, Y. Tian, and Y. Tang, "Large language models as evolution strategies," in *Proc. Genet. Evol. Computation Conf. Companion*, 2024, pp. 579–582.

[60] H. Hao, X. Zhang, and A. Zhou, "Large language models as surrogate models in evolutionary algorithms: A preliminary study," *Swarm Evol. Computation*, vol. 91, 2024, Art. no. 101741.

[61] F. Liu et al., "Large language model for multi-objective evolutionary optimization," in *Proc. Int. Conf. Evol. Multi-Criterion Optim.*, 2023, pp. 178–191.

[62] E. Meyerson et al., "Language model crossover: Variation through few-shot prompting," *ACM Trans. Evol. Learn.*, vol. 4, no. 4, pp. 1–40, 2024.

[63] H. Bradley et al., "Quality-diversity through AI feedback," in *Proc. 12th Int. Conf. Learn. Representations*, May 2024, pp. 1–16. [Online]. Available: https://dblp.org/rec/conf/iclr/Bradley0TZOBCSS24.bib

[64] S. Yao, F. Liu, X. Lin, Z. Lu, Z. Wang, and Q. Zhang, "Multi-objective evolution of heuristic using large language model," in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 25, pp. 27144–27152.

[65] F. Liu, X. Tong, M. Yuan, and Q. Zhang, "Algorithm evolution using large language model," 2023, *arXiv:2311.15249*.

[66] F. Liu et al., "Evolution of heuristics: Towards efficient automatic algorithm design using large language model," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 32201–32223.

[67] N. van Stein and T. Bäck, "LLaMEA: A large language model evolutionary algorithm for automatically generating metaheuristics," *IEEE Trans. Evol. Comput*, vol. 29, no. 2, pp. 331–345, Apr. 2025.

[68] X. Yang, R. Wang, K. Li, W. Li, and W. Huang, "Large language model assisted automated algorithm generation and evolution via meta-black-box optimization," *Expert Syst. Appl.*, vol. 317, 2025, Art. no. 131756.

[69] F. Liu, R. Zhang, X. Lin, Z. Lu, and Q. Zhang, "Fine-tuning large language model for automated algorithm design," 2025, *arXiv:2507.10614*.

[70] A. Chen, D. Dohan, and D. So, "Evoprompting: Language models for code-level neural architecture search," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, vol. 36, pp. 7787–7817.

[71] M. Zheng et al., "Can GPT-4 perform neural architecture search?" 2023, *arXiv:2304.10970*.

[72] R. Qin, Y. Hu, Z. Yan, J. Xiong, A. Abbasi, and Y. Shi, "FL-NAS: Towards fairness of NAS for resource constrained devices via large language models," in *Proc. Asia South Pacific Des. Automat. Conf.*, 2024, pp. 429–434.

[73] H. Dong, Y. Gao, H. Wang, H. Yang, and P. Zhang, "Heterogeneous graph neural architecture search with GPT-4," 2023, *arXiv:2312.08680*.

[74] H. Wang et al., "Graph neural architecture search with GPT-4," 2023, *arXiv:2310.01436*.

[75] S. Zhang, C. Gong, L. Wu, X. Liu, and M. Zhou, "Automl-GPT: Automatic machine learning with GPT," 2023, *arXiv:2305.02499*.

[76] G. Jawahar, M. Abdul-Mageed, L. V. Lakshmanan, and D. Ding, "LLM performance predictors are good initializers for architecture search," in *Proc. Findings Assoc. Comput. Linguistics*, 2023, pp. 10540–10560.

[77] C. Morris, M. Jurado, and J. Zutty, "LLM guided evolution-the automation of models advancing models," in *Proc. Genet. Evol. Computation Conf.*, 2024, pp. 377–384.

[78] X. Zhou, L. Feng, X. Wu, Z. Lu, and K. C. Tan, "Design principle transfer in neural architecture search via large language models," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 23000–23008.

[79] P. Maddigan, A. Lensen, and B. Xue, "Explaining genetic programming trees using large language models," 2024, *arXiv:2403.03397*.

[80] X. Zhou, Z. Wang, L. Feng, S. Liu, K.-C. Wong, and K. C. Tan, "Toward evolutionary multitask convolutional neural architecture search," *IEEE Trans. Evol. Comput.*, vol. 28, no. 3, pp. 682–695, Jun. 2024.

[81] P. Liao, Y. Jin, and W. Du, "EMT-NAS: Transferring architectural knowledge between tasks from different datasets," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 3643–3653.

[82] Y. Liu et al., "Evolutionary multitasking CNN architecture search for hyperspectral image classification," in *Proc. Int. Joint Conf. Neural Netw.*, 2022, pp. 01–08.

[83] X. Zhou, A. K. Qin, M. Gong, and K. C. Tan, "A survey on evolutionary construction of deep neural networks," *IEEE Trans. Evol. Comput.*, vol. 25, no. 5, pp. 894–912, Oct. 2021.

[84] Y. Zhang, P. Tiňo, A. Leonardis, and K. Tang, "A survey on neural network interpretability," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 5, no. 5, pp. 726–742, Oct. 2021.

[85] Y. Tian, T. Zhang, J. Xiao, X. Zhang, and Y. Jin, "A coevolutionary framework for constrained multiobjective optimization problems," *IEEE Trans. Evol. Comput.*, vol. 25, no. 1, pp. 102–116, Feb. 2021.

[86] E. Ziztler, M. Laumanns, and L. Thiele, "SPEA2: Improving the strength Pareto evolutionary algorithm," in *Evolutionary Methods for Design, Optimization, and Control*. Cham, Switzerland: Springer, 2002, pp. 95–100.

[87] E. Alba and B. Dorronsoro, "The exploration/exploitation tradeoff in dynamic cellular genetic algorithms," *IEEE Trans. Evol. Comput.*, vol. 9, no. 2, pp. 126–142, Apr. 2005.

[88] Y. Bi, B. Xue, and M. Zhang, "Genetic programming-based evolutionary deep learning for data-efficient image classification," *IEEE Trans. Evol. Comput.*, vol. 28, no. 2, pp. 307–322, Apr. 2024.

[89] M. Feng, N. Heffernan, and K. Koedinger, "Addressing the assessment challenge with an online system that tutors as it assesses," *User Model. User-Adapted Interaction*, vol. 19, no. 3, pp. 243–266, 2009.

[90] Y. Lu, Y. Pian, Z. Shen, P. Chen, and X. Li, "SLP: A multi-dimensional and consecutive dataset from K-12 education," in *Proc. Int. Conf. Comput. Educ. Conf.*, 2021, vol. 1, pp. 261–266.

[91] USTC, "Frusub dataset," 2021. [Online]. Available: http://staff.ustc.edu.cn/%7Eqiliuql/data/math2015.rar

[92] M. O. Z. S. Pedro, R. S. d Baker, S. M. Gowda, and N. T. Heffernan, "Towards an understanding of affect and knowledge from student interaction with an intelligent tutoring system," in *Proc. Int. Conf. Artif. Intell. Educ.*, 2013, pp. 41–50.

[93] S. Yang et al., "Endowing interpretability for neural cognitive diagnosis by efficient Kolmogorov-Arnold networks," in *Proc. 34th Int. Joint Conf. Artif. Intell.*, Montreal, Canada, Aug. 2025, pp. 7894–7903, doi: 10.24963/IJCAI.2025/878.

[94] K. Xu, X. Shao, Y. Tian, S. Yang, and X. Zhang, "AutoMPQ: Automatic mixed-precision neural network search via few-shot quantization adapter," *IEEE Trans. Emerg. Topics Comput. Intell.*, early access, May 08, 2024, doi: 10.1109/TETCI.2024.3394679.

[95] R. Mohapatra, S. Saha, C. A. C. Coello, A. Bhattacharya, S. S. Dhavala, and S. Saha, "AdaSwarm: Augmenting gradient-based optimizers in deep learning with swarm intelligence," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 6, no. 2, pp. 329–340, Apr. 2022.

[96] H. Gao, X. Qin, R. J. D. Barroso, W. Hussain, Y. Xu, and Y. Yin, "Collaborative learning-based industrial IoT API recommendation for software-defined devices: The implicit knowledge discovery perspective," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 6, no. 1, pp. 66–76, Feb. 2020.

[97] T. Van Belle and D. H. Ackley, "Uniform subtree mutation," in *Proc. Eur. Conf. Genet. Program.*, 2002, pp. 152–161.

[98] H. Zhang, Q. Chen, B. Xue, W. Banzhaf, and M. Zhang, "A semantic-based hoist mutation operator for evolutionary feature construction in regression," *IEEE Trans. Evol. Comput.*, vol. 28, no. 6, pp. 1689–1703, Dec. 2024.

[99] Y. Liu, J. Liu, Y. Jin, F. Li, and T. Zheng, "A surrogate-assisted two-stage differential evolution for expensive constrained optimization," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 7, no. 3, pp. 715–730, Jun. 2023.

Authorized licensed use limited to: Anhui University. Downloaded on October 26,2026 at 14:55:52 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.



Shangshang Yang (Member, IEEE) received the B.Sc. and Ph.D. degrees from Anhui University, Hefei, China, in 2017 and 2022, respectively. He was a Visiting Ph.D. Student with Bielefeld University, Bielefeld, Germany, in 2022. He is currently a Lecturer with the School of Computer Science and Technology, Anhui University, Hefei, China. His research interests include evolutionary multi-objective optimization, neural architecture search, intelligent education, and graph learning. He was awarded the Postdoctoral Fellowship Program (Grade B) of China Postdoctoral Science Foundation. He was the recipient of the 2023 International Conference on Data-driven Optimization of Complex Systems Best Paper Award.



Linrui Qin received the B.Sc. degree in 2021 from Anhui University, Hefei, China, where he is currently working toward the M.Sc. degree. His research interests include large language models, post-training, reinforcement learning from verifiable rewards, and neural architecture search.



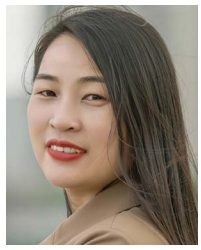
Xiaoshan Yu is currently working toward the Ph.D. degree with the School of Artificial Intelligence, Anhui University, Hefei, China. His research interests include intelligent education, multi-objective optimization, neural architecture search, and graph learning.



Ziwen Wang received the B.S. degree in 2020 from Anhui University, Hefei, China, where he is currently working toward the Ph.D. degree with the School of Computer Science and Technology. His research interests include multi-objective optimization, neural architecture search, and trustworthy intelligent education.



Ye Tian (Senior Member, IEEE) received the B.Sc., M.Sc., and Ph.D. degrees from Anhui University, Hefei, China, in 2012, 2015, and 2018, respectively. He is currently a Professor with the School of Computer Science and Technology, Anhui University. His research focuses on evolutionary computation and its applications. He was the recipient of the 2018, 2021, and 2024 IEEE Transactions on Evolutionary Computation Outstanding Paper Award, 2020 IEEE Computational Intelligence Magazine Outstanding Paper Award, and 2022 IEEE Computational Intelligence Society Outstanding PhD Dissertation Award.



Xueming Yan (Senior Member, IEEE) received the Ph.D. degree in computer science from the South China University of Technology, Guangzhou, China, in 2017. She was a Visiting Scholar with the University of Surrey, Guildford, U.K., in 2021 and Postdoctoral Researcher with Bielefeld University, Bielefeld, Germany, from 2022 to 2023. She is currently a Professor with the School of Information Science and Technology and School of Cyber Security, Guangdong University of Foreign Studies, Guangzhou, China. Her research interests include intelligent optimization, neural architecture search, and AI systems for multilingual, multimodal, and crosscultural applications. Dr. Yan is an Associate Editor for *Complex & Intelligent Systems* and *Neurocomputing*.



Haiping Ma received the B.E. degree from Anhui University, Hefei, China, in 2008, and the Ph.D. degree from the University of Science and Technology of China, Hefei, in 2013. She is currently a Lecturer with the Institutes of Physical Science and Information Technology, Anhui University. Her research interests include data mining and multiobjective optimization methods and their applications.



Xingyi Zhang (Fellow, IEEE) received the B.Sc. degree from Fuyang Normal College, Fuyang, China, in 2003, and the M.Sc. and Ph.D. degrees from the Huazhong University of Science and Technology, Wuhan, China, in 2006 and 2009, respectively. He is currently a Professor with the School of Computer Science and Technology, Anhui University, Hefei, China. His research interests include unconventional models and algorithms of computation, evolutionary multi-objective optimization, and logistic scheduling. He was the recipient of the 2018, 2021, and 2024 IEEE Transactions on Evolutionary Computation Outstanding Paper Award and 2020 IEEE Computational Intelligence Magazine Outstanding Paper Award.

Authorized licensed use limited to: Anhui University. Downloaded on October 26,2026 at 14:55:52 UTC from IEEE Xplore. Restrictions apply.